

B.Tech Project Report

- ❖ Development of Online Computer based Test Suite, to be deployed on www.uCertify.com
- ❖ Development of a generic ASP based toolkit for remote database manipulation, with intermediate data transfer in XML

Project Guide

Mr. Madhup Sharma
Senior Project Manager
Ucertify.com

Project Undertaken By

Pratyush Kumar (19991034)
Pravi Tripathi (19991035)

ACKNOWLEDGEMENT

We express our sincere gratitude and thanks to all those who have helped us in the completion of this project.

Of all the persons who have helped us, we would first of all like to thank Mr. Madhup Sharma, Senior Project Manager (uCertify.com), under whose able guidance we have completed our project and who helped us at each and every stage of our project.

We would also like to extend our gratitude to Mr. Prashant Gupta, CEO, uCertify.com and his team Ms Priya, Mr. Varun, Mr. Deepak Jaiswal and everybody else at uCertify.com for their cooperation and valuable support in every way during the period of the project.

Pratyush
Pravi Tripathi

Eighth Semester
IIIT-Allahabad

Company Profile: www.uCertify.com

Conceived in 1996, uCertify.com specializes in the development of computer assisted test preparation software. From the very beginning, the company has devoted considerable resources in Computer Assisted Learning (CAL) research. The findings from these researches have been incorporated into their products. Hence these products are not only effective in providing a realistic test simulation but also help preparing for the certification tests.

The company covers major internationally recognized computer certification tests such as

- 1. MCSE**
- 2. MCSA**
- 3. MCDBA**
- 4. MCSA**
- 5. CompTIA**
- 6. Java Certification**
- 7. CIW**
- 8. Oracle**

Their flagship project uC PrepEngine and uC PrepKit has already registered widespread consumer base just during the first few months of its release. The team is committed to enhance its knowledge base to contribute constructively towards facilitating aspiring students of these certification exams. Further the company plans to enter into the huge market of online testing services, chiefly for the US, like the online nursing test and GRE/GMAT to name a few.

The company also undertakes several offshore projects from companies abroad. The projects usually harness the strong skill set of the team chiefly in the areas of databases, portal design and general software consultancy.

Their major products include the uCertify PrepEngine and the uCertify PrepKit. PrepEngine is a generic engine that has been written to read test data from an installed PrepKit to simulate practice tests.

Assignment I

To Develop a Online Test Suite, to be deployed on www.uCertify.com

Time Line

Problem Analysis	10%
Study	30%
Module Division	5%
Design	10%
Implementation Debugging and Integration	40%
Project Report and Presentation	5%

Table Of Content

1. Abstract
2. Introduction
3. Problem Definition and Analysis: requirements
4. Design and Implementation
5. Testing and Deployment
6. Future Enhancements
7. Bibliography

Abstract

Companies existing on the web have several constraints. The primary constraint with such companies is the way they interact with their customers. The customer has to have an internet access for sufficiently long time before they can really impress her and sell their product. Companies long for a solution that can be viewed offline with almost same features as online ones should be small in size and should protect their IPR. We have here tried to explore a solution for such a scenario. The company, uCertify.com has been in the business of selling online test preparation kits and wants a solution to reach to its customers online as well as offline.

We propose the use of web technologies like DHTML and XML for designing such a solution. The solution thus designed has the advantage of being used into their flagship product uCertify PrepEngine and the PrepKit to render it more flexibility and extensibility.

Introduction

Since advent of the computer aided learning paradigm, many companies have started their online courses related to their technologies and products. To name a few are like Cisco Certified Networking Administrator (CCNA), Microsoft Certified Systems Engineer (MCSE) and Microsoft Certified Software Developer (MCSD). These companies give a certification subject to clearing all of their online tests under certain category. To cater the need of this growing market segment, where online test preparation kits hold a great popularity, uCertify has been in the forefront, developing simulation test preparation kits.

We as working with their team of creative professionals were given a task to develop an online test kit on the lines of Microsoft, along with careful analysis of competitors software's for the similar purpose, so that we could create a place for uCertify's software products. Major constraints before us were performance along with reliability i.e. a user should not experience any glitches and error messages while undergoing the online test. Also the interface along with the usage procedures had to be as simple and user friendly as possible.

In such kind of application user interface design plays an important factor when one has to compete with various other products of different companies. The user will definitely opt for the one he feels more comfortable and user friendly rather than one with complex features and handling issues.

Problem Definition and Analysis: Requirements

uCertify.com has its major business area in marketing simulation tests for Microsoft and other similar computer based tests. Hence the project requirements were to simulate similar test experience as on the Microsoft online tests. There were many categories of questions that Microsoft corp. asks. All such types of questions were to be included in the online test module. Also, the company decided to keep this module for promotional purposes on the internet as well. The company wanted a solution that could serve as a general solution for all its requirements, the web, the uCertify PrepEngine and even for promotional purposes application. It was to function as a generic solution for the migration of company's dependence on rigid test application made with regular programming languages, such as PowerBuilder and Sybase as backend database towards a DHTML style. Hence, an important consideration of small size and light weight application was imperative. Care was to be taken so that the application developed could work with a wide variety of systems and platforms, ranging from legacy windows systems, to the most modern Windows XP, to other third party operating systems and browsers.

Major requirements can be listed as:

1. Light weight application
2. Single file that could easily be downloaded even on slow connections
3. Should work on various third party browsers (online web page version)
4. Should be able to use company's existing question database with minimum changes made
5. Extensible
6. Should incorporate major types of questions asked by Microsoft Corp.

Depending on the requirements of the test, a PrepKit can have following types of questions:

1. Single Correct in Multiple Choice
2. Multiple Correct in Multiple Choice
3. Scenario-Based
4. Exhibit-Based
5. Hot Spot
6. Drag & Drop

7. Choose and Reorder the List
8. Create a Tree
9. Establish a Relationship

Single Correct in Multiple Choice

The Single correct in multiple choice type questions have multiple answer options with radio buttons. Radio buttons indicate that only one answer option can be selected. This does not mean in anyway that only the selected answer option is correct. All the answer options may or may not be correct, but only the most appropriate answer option is to be selected.

Example:

- ☒ A. Answer option 1
- ☐ B. Answer option 2

Multiple Correct in Multiple Choice

The Multiple correct in multiple choice type questions have multiple answer choices with check boxes. Check boxes indicate that there can be one or more correct answers for that question. If the question requires three answers and only two or less than two are marked, the question will be marked as incorrect.

Example:

- ☐ A. Answer option 1
- ☐ B. Answer option 2

Scenario-Based

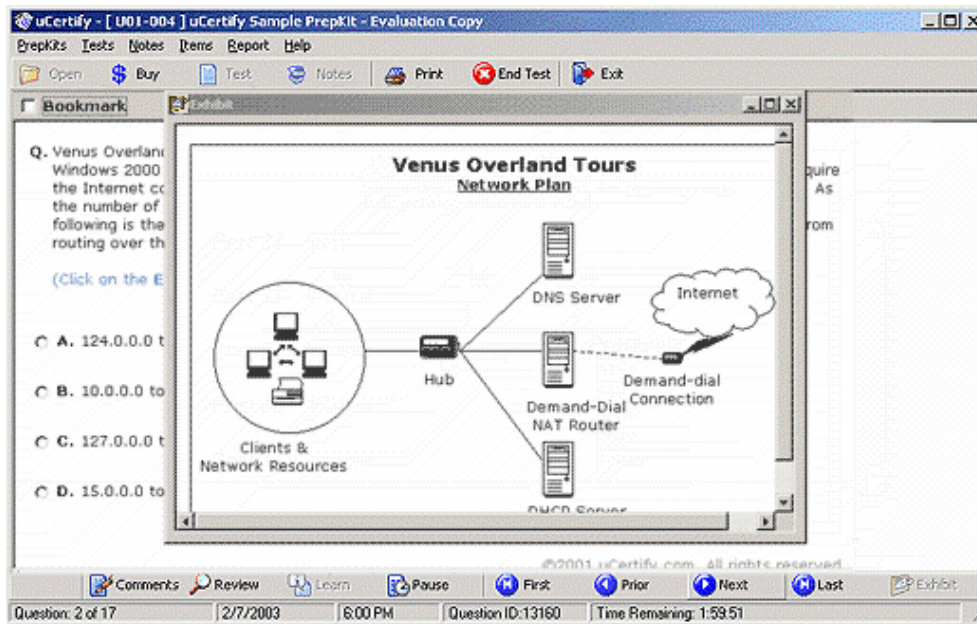
Scenario-based questions are based on scenarios and exhibits. The question requires the test taker to rate the suggested solution in terms of its aptness in meeting the required and the optional results mentioned in the question.

Example of the Choices:

- ☐ A. The suggested solution does not produce the required result.
- ☐ B. The suggested solution produces only the required result.
- ☐ C. The suggested solution produces the required result and both the optional results.
- ☐ D. The suggested solution produces the required result and one of the optional results.

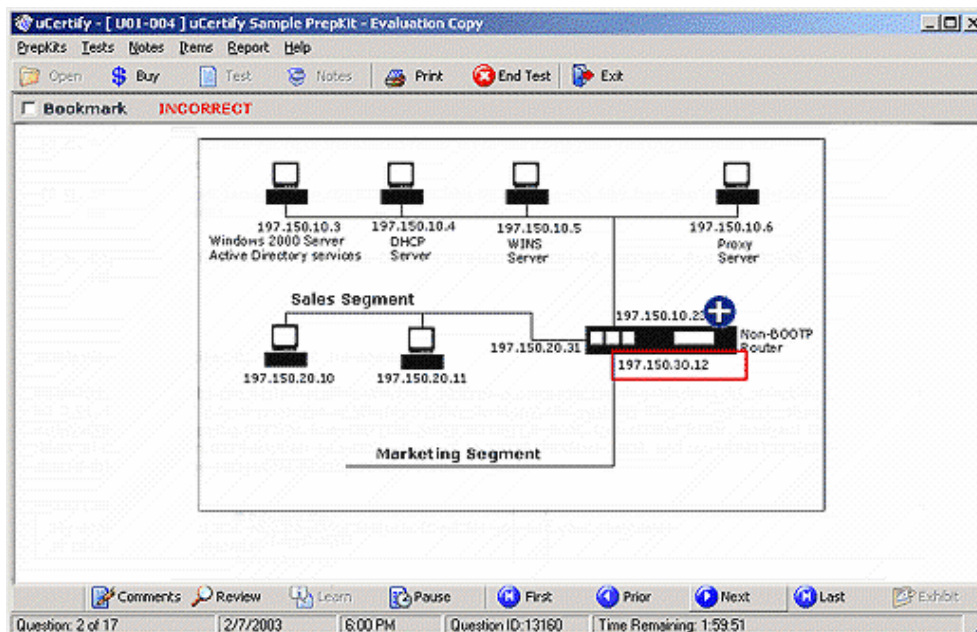
Exhibit-Based

Some questions include exhibits that act as supporting pictorial information for the question. These exhibits are generally used to explain the scenario in the question and help the test taker to understand the question. The **Exhibit-Based** questions may have single correct or multiple correct answers or may even require the test taker to Drag & Drop the correct answer in the placeholder provided, or simulate the question in the exhibit, or click on the Hot Spot in the exhibit. Other types of questions, which may have an exhibit, are Choose and Reorder the List, Create a Tree and Establish a Relationship.



Hot Spot

In Hot Spot type questions, the test taker will be required to mark the appropriate area on the image in the Hot Spot window.

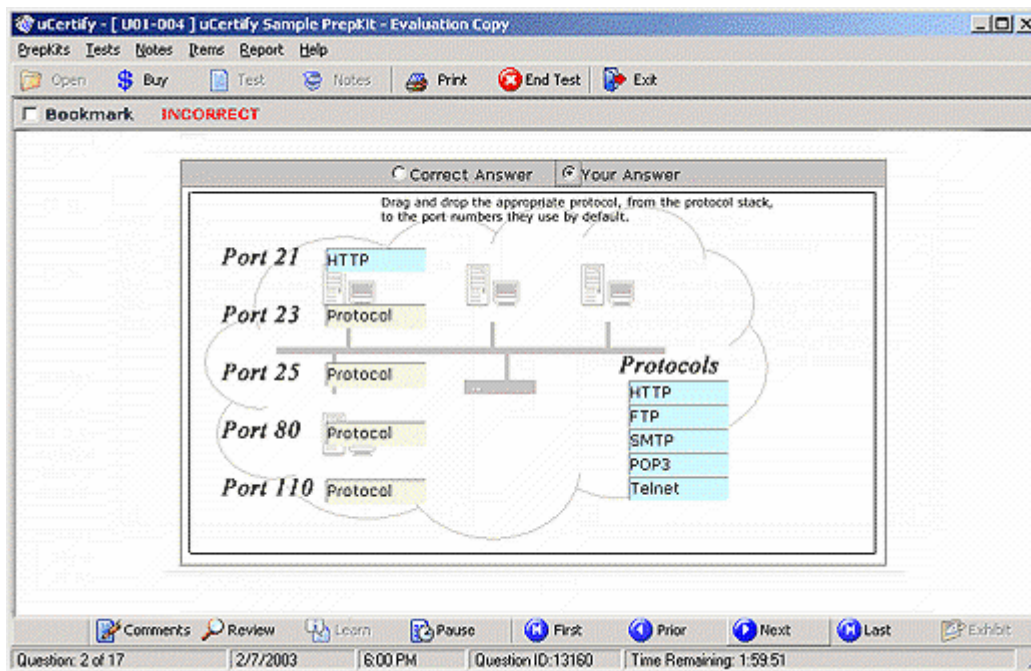


Drag & Drop

To answer Drag & Drop type of questions, the test taker must drag the correct item from the list of items given, and place them in the given placeholder area. To remove the already placed object, drag n drop it anywhere outside the placeholder area. This action brings the item back to the list of items.

Question grading:

One mark is awarded for these types of questions. These marks are awarded if all the items are placed correctly. No marks are awarded if any one of the items is placed incorrectly.



Choose and Reorder the List

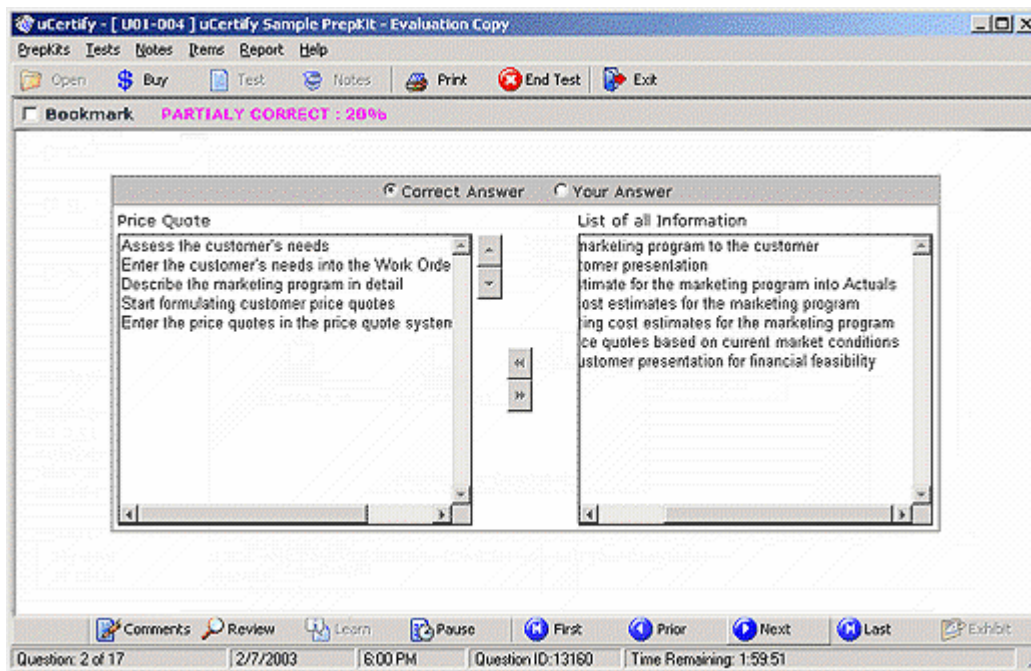
To answer Choose and Reorder the List type of questions, the test taker has to select the correct items from the list of choices on the right hand side of the screen and bring them to the left hand side of the screen. To bring items, one by one, to the left hand side of the screen, click on the item in the right hand side of the screen and then click the left arrow button.

Once the items have been selected and brought to the left hand side of the screen, they can be rearranged as desired. To rearrange the list of answers, one has to select the items to be moved up or down, and then use the up or down arrow button to move it.

To remove an item from the left side screen, select the item and click the right arrow button.

Question grading:

Two points are awarded for each correct item in the list of answers. One point is awarded for each item that appears in the correct order with respect to the other correct items in the list of answers. Two points are deducted for each incorrect item in the list of items. The minimum score is zero



Create a Tree

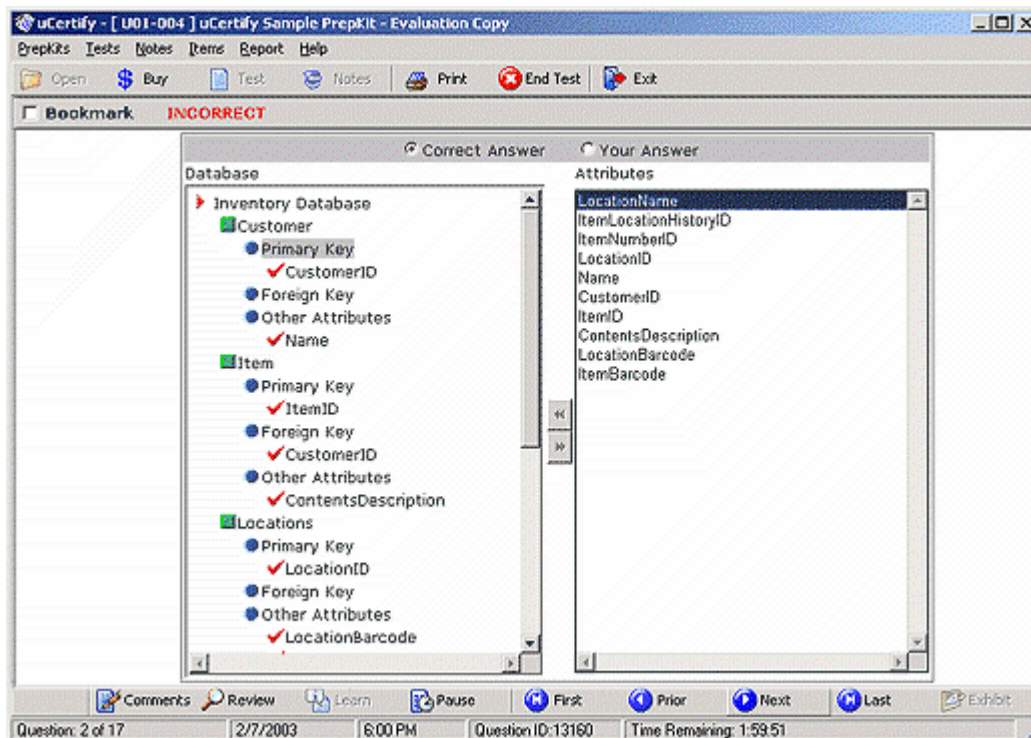
To answer Create a Tree type questions, the test taker must select the appropriate item from the list of items on the right side of the screen and place them under the node of the tree.

To add Items under the nodes, she should select the destination node on the left side of the screen, select the item from the list of items on the right side of the screen, and then click the left arrow button to move the items.

To remove an item from the tree, she can select the item and click the right arrow button, or she can drag the item from the tree back to the list of items on the right side of the screen.

Question grading:

Two points are awarded for each item that appears under the correct node in the tree. One point is deducted for each incorrect item. The minimum score is zero



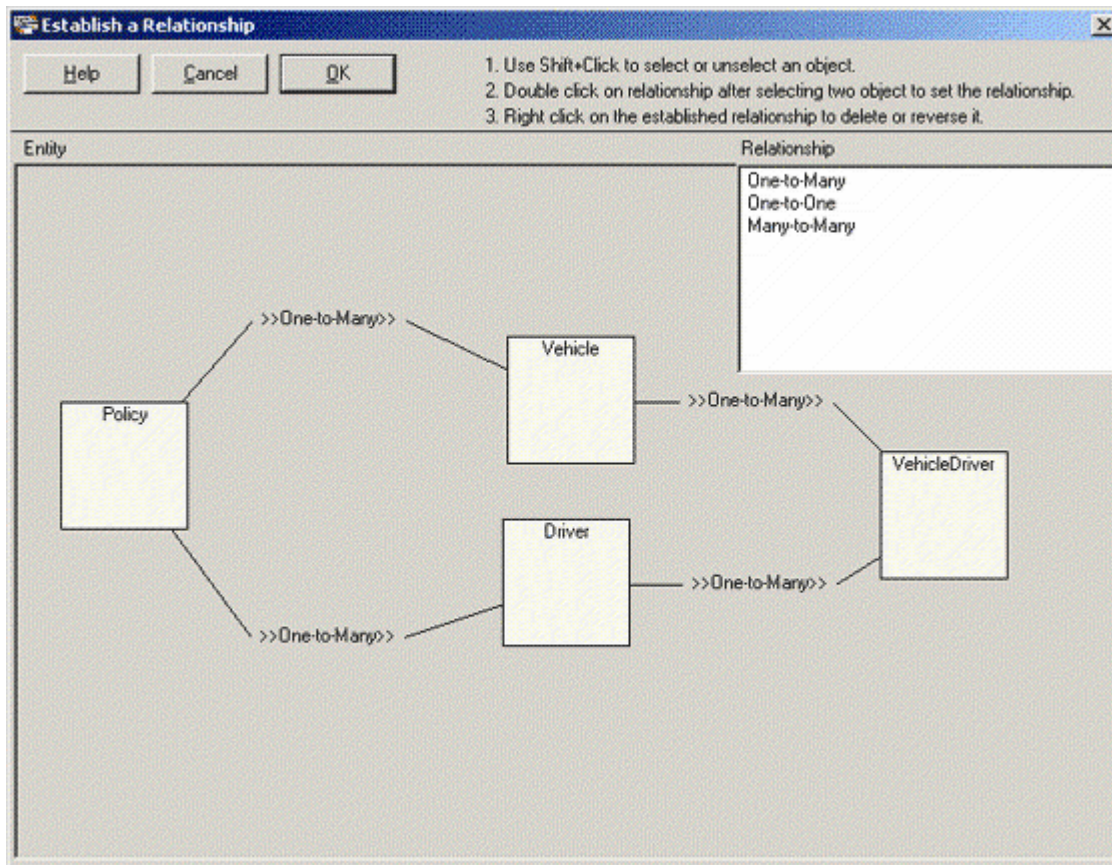
Establish a Relationship

These type of questions present the test taker with several connectors that she can use to join the objects on the screen. Depending on the question, she should connect the objects using the appropriate connector from either the Relationships box or the Data connector box.

To create relationships between two objects, she should first drag the two objects to an open area of the screen. Next, use shift + click to select two objects she wants to connect. Once she has selected the object, the object border changes and shows a 3D-lowered effect. To deselect the object, point to the object and use shift + click again. She must select two objects before she can choose a connector to create a relationship. While defining relationships only two objects should be selected at one time. If two objects are already selected and the test taker wants to select another object, she must first deselect one of the objects. After selecting the two objects, double-click the connector she wants to place between the two objects. She can place a maximum of three connectors between two selected objects. To remove a connector between two objects, she must right-click the connector she wants to remove. Right clicking brings up a pop-up menu from which she can select the Delete option to remove the relationship connector. To reverse the defined relationship of the connector, right-click the connector and select the Reverse option. Simply moving two objects on the screen will not change the relationship.

Question grading:

Each correct relationship between two objects is worth four points. One point is awarded for each correctly selected object that exists in the relationship. One point is given for identifying the correct relationship, and one point for identifying the correct direction of the relationship. Two points are deducted for each incorrectly identified relationship between two objects. The minimum score is zero.



Design and Implementation

Ucertify has long been in the area of online testing. The earlier solution was based on a database front-end application, using Powerbuilder and Sybase SQL Anywhere as the database. The new strategy of the company was to migrate from this rigid application towards a more general and more flexible approach, which could be used simultaneously on the web as well as in the UC PrepEngine. After reviewing the power of the standard tools available, an approach based on XML and HTML was suggested. The new design of the PrepEngine uses IE placeholder to display the web contents, hence additional need for UI programming is obliterated, at the same time providing a single solution for application as well as web requirements of the company. This has been a big paradigm shift for the company.

The major advantage of using HTML with XML as data source is particularly derived from the fact that XML provides generality of data as well as support for its formatting, that's paramount for a good user experience. The use of HTML as the front end provides excellent integration with powerful scripting languages such as Jscript (or JavaScript) and VBScript. The seamless integration with the DOM (Document Object Model) standard also lends it extra ease of Object Oriented programming.

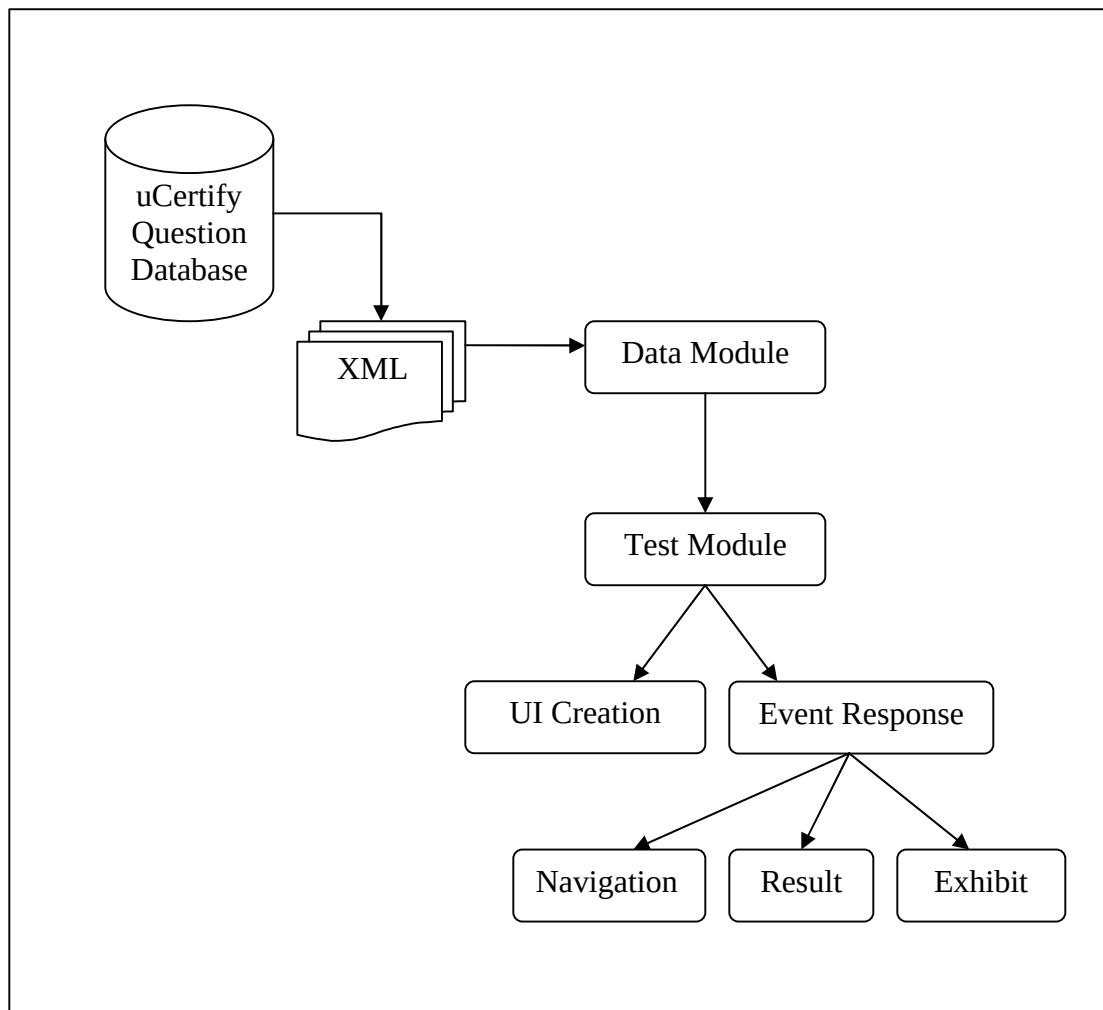
Before the introduction of HTML+XML approach, the company targeted test-takers only interested in Microsoft certification exams, such as the MSCE and MCSP etc. with the migration of the platform to HTML+XML standard, the company can now cater to the needs of a wide variety of users, often not interested only in Microsoft certification. As XML and HTML are general and platform independent implementations, they are pretty well supported on a wide variety of systems.

Another problem now addressed by the XML approach was the ease of data upgrade on client machines. Earlier, the data was distributed in form of MS .mdb files that were registered with the ODBC to serve data to the PrepEngine. With the release of a new PrepKit, it was difficult to upgrade just the database. All the dependencies were first to be closed before any replacement of the database was done. It was also difficult to download just the database from the internet and update the database. With the XML database, it becomes relatively simple to download and update the database. Also the size of the bundled package reduces considerably as the .mdb database required ODBC drivers for those machines that did not have proper mdb drivers installed in them. No such dll's or drivers are required with XML files. They are easier to access using the MS XML parser bundled along with Internet Explorer.

Consequently, the main aim was to migrate the existing data from RDBMS to an XML file. We worked separately on this module, providing extra functionality of exporting files of a remote database, through a web server and capturing it in form of XML.

The conversion of this RDBMS to XML required additional design issues that could faithfully depict the relations and hierarchy that was already stored in the database.

The solution designed can be explained with the help of the following block diagram.



The company decided to keep the online test application on the web for promotional purposes. Making a rigid application would hamper the size considerations. Also, just keeping a webpage on the web for testing requires the

test taker to be online for a considerable period of time. Considering the large variety of test takers, this was a big assumption to make. Hence, after considering all the possible alternatives, we finally settled on developing a single module that could function as a standalone test, but light weight in size. An application with all its dll's is demanding in size. The best alternative was a Compiled Help Module, or commonly called a CHM file. A CHM file is a Microsoft standard for making HTML help for applications, and is very widely used by software vendors to include product help. The file being just a single one, is easy to download on the web for offline viewing by a prospective client. Also, the CHM compiler compresses the data while compiling, thereby reducing the size further. All the facilities available in scripting languages work effectively in such file, with extra functionality of full text search and index. Although full text search and indexes were not required with this application, nevertheless, it is an important tool with the company can enhance the user experience of their online test module in future, whenever desired so.

A compiled help module also hides the coding from an end user, thus safeguarding commercial aspirations of the company from its competitors.

Database to XML Conversion:

uCertify uses a database to keep track of questions related to a particular exam paper, its possible options, correct options and other related details. Related details of a question can vary and can potentially be quite large in number. Such details include the type of question, telling the PrepEngine if the question is a single option type, multi-option type, an exhibit based or a special module, or any other question type formed by Microsoft in its exam pattern, the major types of questions can be listed as:

1. Single option Radio Button
2. Multiple answer check box
3. special module type
 - a. Drag Drop type
 - b. HotSpot type
 - c. Choose and Reorder
 - d. Tree View type
 - e. Relationship
4. Exhibit type

The exhibit type might again be any of the other type, with an extra picture or case study as an exhibit.

Other information also includes the exhibit name and its type, if any, the number of answers, and number of choices.

Also, a separate database takes care of storage of respective answer options and the correct ones in them. The database also stores whether a question is a normal question, with a single/multiple answer or is a question of special module. The demarcation of special module questions is essential as its comparatively challenging to implement these questions and check their answers. For example, there is no single and efficient method of checking special module of relationship forming, that is normally asked in database table relationship questions. The user might choose potentially an infinite combination from the displayed combination of databases on the screen, and can relate them in another infinite ways, though such infinite choices normally do not arise with limited databases displayed, the choices do multiply up to large numbers. User answer checking does become a challenge under these circumstances.

There are additional problems associated with the keeping of the choices and correct options in questions relating to choose and reorder type and the treeview type of questions.

Following is the snippet of a node of the XML file, company now uses as the database for online testing.

```
- <QUESTIONS PAPER_CODE="M70-100" PAPER_NAME="Analyzing Requirements  
and Defining Solution Architectures">  
"  
- <QUESTION UCID="0004135" CASEID="0000000" IMG="" Q_TYPE="0">  
  - <DETAIL>  
  - <![CDATA[  
You are working on an application using Visual Basic as the front-end and  
SQL Server as the back-end. Both, Visual Basic and SQL Server, run on  
the same machine. Which application model does this application use?  
  ]]>  
  </DETAIL>  
- <ANSWERS>  
  - <ANSWER UCID="0012492" CORRECT="0" USERANS="0">  
  - <![CDATA[  
N-tier  
  ]]>  
  </ANSWER>  
  - <ANSWER UCID="0012490" CORRECT="0" USERANS="0">  
  - <![CDATA[  
Single-tier  
  ]]>  
  </ANSWER>  
  - <ANSWER UCID="0012493" CORRECT="0" USERANS="0">  
  - <![CDATA[
```

This type of application cannot be written using the current technology.

```
]]>
</ANSWER>
- <ANSWER UCID="0012491" CORRECT="1" USERANS="0">
- <![CDATA[
Two-tier
]]>
</ANSWER>
</ANSWERS>
- <SMUSERANS>
- <![CDATA[
]]>
</SMUSERANS>
- <SMCORRECTANS>
- <![CDATA[
]]>
</SMCORRECTANS>
- <ANALYSIS>
- <![CDATA[
Although the tiers run on the same machine, they are conceptually
separated. Thus, two-tier application model is used in the given
scenario maintainability, as the business rules can be modified
without having to modify the other layers of the application.
border="0"> <tbody> <tr><td height="2"></td> </tr>
</tbody></table><br>
<div class = "reference">Ref: MSDN, Topic: "Three-tier Application
Model"</div>

<<<<"THE COMPLETE ANALYSIS IS NOT GIVEN HERE IN THE REPORT DUE TO
COMPANY POLICY">>>>

]]>
</ANALYSIS>
</QUESTION>
```

Following tags and attributes are used to store relevant information in the XML file

1. PAPER_CODE: Microsoft specified code name of a particular test
2. PAPER_NAME: stores the name of a particular paper.

3. UCID: unique ID of a question. It corresponds with the primary key on the UC Question database. This field helps identify a question uniquely so that it can be referenced any time desired.
4. CASEID: stores the name of the case study to be displayed with a question, if any. If none, this field is a string of 0's.
5. IMG: stores image exhibit. NULL if none
6. Q_TYPE: stores type of a particular question that identifies it as a single answer type, multiple answer type, or one of the special module, etc.
7. DETAIL: stores the question
8. ANSWERS: a structure that hierarchically saves all the answer options, their UCID and whether which one of them is correct.

```

- <ANSWERS>
- <ANSWER UCID="0012492" CORRECT="0" USERANS="0">
- <![CDATA[
N-tier
]]>
</ANSWER>
- <ANSWER UCID="0012490" CORRECT="0" USERANS="0">
- <![CDATA[
Single-tier
]]>
</ANSWER>
- <ANSWER UCID="0012493" CORRECT="0" USERANS="0">
- <![CDATA[
This type of application cannot be written using the current technology.
]]>
</ANSWER>
- <ANSWER UCID="0012491" CORRECT="1" USERANS="0">
- <![CDATA[
Two-tier
]]>
</ANSWER>
</ANSWERS>
- <SMUSERANS>
- <![CDATA[
]]>
</SMUSERANS>
- <SMCORRECTANS>
- <![CDATA[
]]>
</SMCORRECTANS>

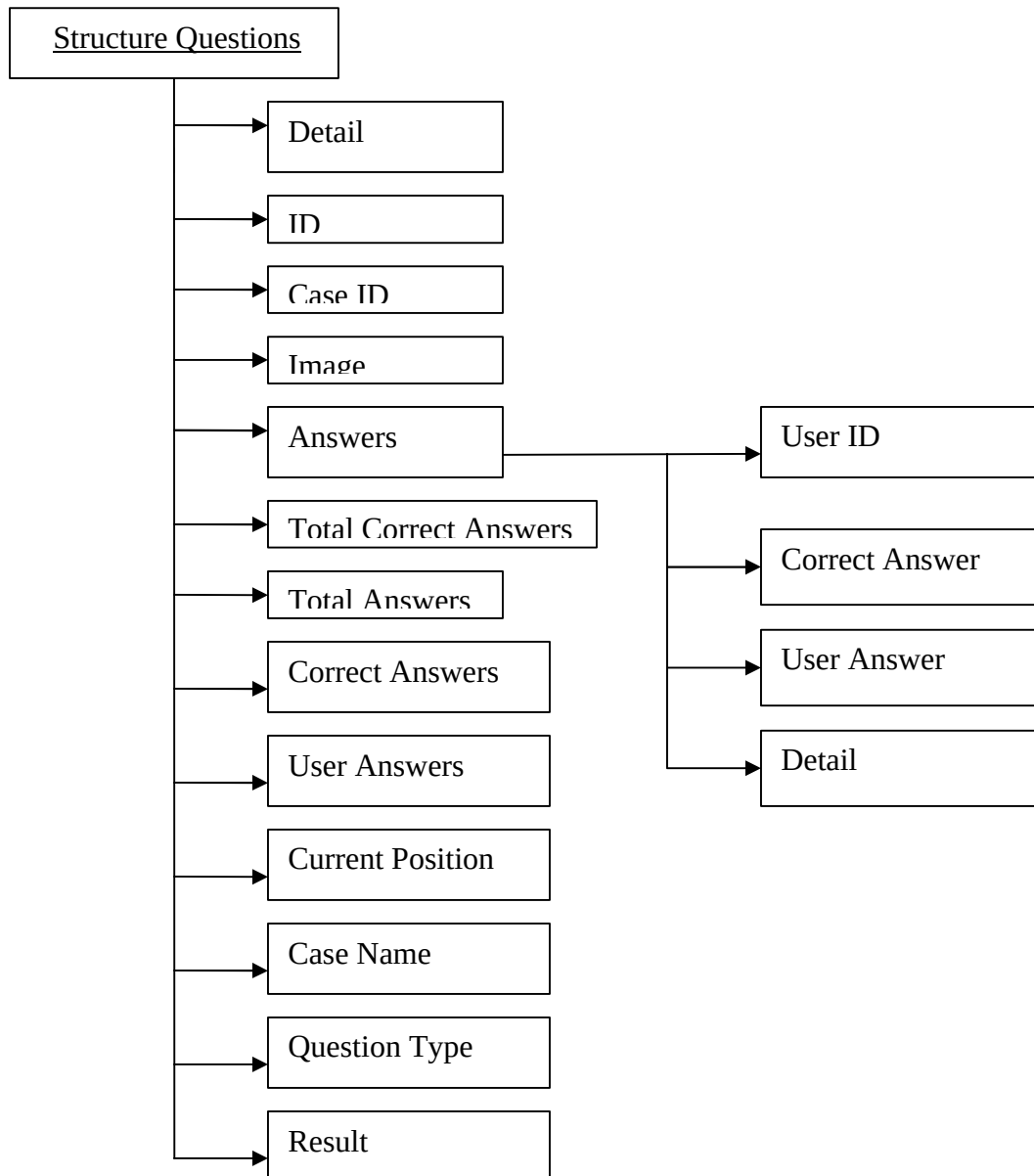
```

9. SMUSERANS: stores the snapshot of user's interaction with a particular question, so that it might later be reproduced when working in review mode.
10. SMCORRECTANS: stores the correct answer of a question. Used when working in review mode to generate snapshot of correct answer.
11. ANALYSIS: stores the analysis that is to be provided with a question when working in the review mode.

Data Module:

The data module takes the responsibility of parsing the XML database provided into a runtime data structure. This data structure generated during runtime is then used throughout the interaction scenario. All the user events are captured into this data structure and the same data structure is then used when working in review mode or result generation time. When working in the review mode, the user is allowed to see his answer of a question, and the correct answer side by side so that he may compare and see his mistakes. The analysis of the question is also displayed at that time. All these fields need to be read and fed into a database so that they may be generated on runtime.

The hierarchy of the database can be depicted diagrammatically as:



The structure of the data structure designed can be explained by the following code snippet:

```
function question()  
{  
    this.detail;  
    this.id;  
    this.caseid;  
    this.img;
```

```
        this.answers;
        this.totalcorrect;
        this.totalans;
        this.analysis;
        this.smcorrectans;
        this.smuserans;
        this.smccurrentpos;
        this.smccurrentpos1;
        this.casename;
        this.q_type;
        this.result;
    }
```

This is a structure declared in JavaScript. All the fields correspond to the fields of the XML database.

Answers are stored in the subfield answers, which are further declared at runtime to be a structure of the type ans. The structure **ans** can be defined as:

```
function ans()
{
    this.id;
    this.correct;
    this.userans;
    this.detail;
}
```

These, then form array of structure during the runtime to keep track of questions and their interaction with the user. Note that all these data elements declared are not specifically typecast. The type of these elements is decided by JavaScript at the run time.

Test Module:

The test module is the one dealing with generation of the online test and user interaction. It can be divided into following main submodules:

1. Test module
 - a. UI Creation
 - b. Event capture
 - i. Navigate event
 - ii. Show exhibit event
 - iii. Result generation
 - iv. Review mode

UI Creation:

The user interface created depends on the type of question asked. If it is a single answer type question, the options must accompany radio button grouped together so that only a single selection can be made. If it is a multiple answer type question, check boxes need to be made. Complex measures are adopted if any of the special module questions are to be asked. The UI creation module also takes care of initializing the user answer datastructure so that any interaction can faithfully be reproduced when navigating back and forth or in review mode. Also, steps need to be taken to evaluate user response depending upon the type of question asked.

Further implementation details shall be discussed according to the different question scenarios.

Event Capture:

This submodule deals with the interpretation of user response to a question. The events are broadly divided into 4 categories. The navigate event essentially maintains a state variable to keep track of current question.

The variable `curq` holds value of current question. This variable is incremented or decremented depending on the direction of navigation. The following code snippet demonstrates the loading of a new question based on the present value of the variable `curq`.

```

var rc;
rc = curq + 1;
pos.innerHTML = rc + "/" + totalq;
    var uans,casedetail;
    casedetail = "";
    pagecontent = "<table border=0><tr><td id=qresult>&nbsp;</td></tr></table>"
    if (questions[curq].casename != "")
    {
        casedetail = '<a href="'+ "c"+questions[curq].caseid+".htm" + "'
target="_blank">'+questions[curq].casename+'</a>'
    }
    pagecontent = pagecontent + "<table border=0><tr><td>"
+casedetail+"</td></tr></table>"
    pagecontent = pagecontent + "<div class=question><b>Q: </b>" +
questions[curq].detail + "</div>";
    pagecontent = pagecontent + specialinstruntction(questions[curq].q_type);
    pagecontent = pagecontent + '<FORM name = "testform"
ACTION="&[documenturl];action=show" METHOD="POST"><table>';
    for (var i = 0; i < questions[curq].totalans; i++)

```

The result generation and review mode both work in similar manner. A Boolean state variable identifying whether the test is in "test" mode or "review" mode is set true to signify review mode. When working in the review mode. Answer options are disabled and analysis is displayed along with a question. Also, there is an option to see correct solution or the solution given by the user. All these values are fetched from the data structure that keeps track of user activity with a particular question. The information about user moves is stored in the data structure called **ans.userans** which is used to display when working in the review mode or during navigation.

The show exhibit event occurs when there is either a case study or an exhibit associated with a question. The name of the case study or image is given in the xml tags with same name. When a show exhibit event occurs, these data structures are searched for adequate information, and if present, they are fetched and displayed. Another thing to notice is that the even show exhibit is tied down to the **onclick** event of a button at the bottom corner of the screen, and it is disabled if the question provides no exhibit or case study.



The UI module and the test module simultaneously implement functions that deal with all kinds of question scenarios that can occur in an online test. A

general use case scenario occurs when a particular type of question is to be drawn on the user interface and the user's responses need to be evaluated. Consequently separate functions were written to take care of these needs of the online test application. The design and implementation of these scenarios can further be explained as under.

Single answer type questions:

The single answer type questions are the simplest ones to implement, check and keep track of. There were particularly minimal challenges posed by such questions. The XML database straight away provides options for such questions, and also indicates which one is the correct. An important step is that of distinction between single answer type and multiple answer type. Counting the number of correct answers can easily do that. This number is stored in the **questions** data structure member **totalcorrect**.

For the user interface design, such questions need a radio button drawn in front of each option. The **navigate** event of each question page captures the snapshot of user action in the **userans** structure which can later be used to evaluate the result in the end, or whenever demanded. No computations are done while a question is being answered, chiefly for the following reasons.

1. A user might return back to an answered question before he finishes the test. Calculation each time he clicks only tends to make the test module slow on lower end machines.
2. Additional code needs to be written that is unnecessarily complex and deals with repeated answer array parsing, which increases temporal complexity of the online test. Instead of making computations at each step, the options are stored and all computations are done in the end, so the user has to wait only once, and not at each click. This results in a better user experience.

```

        if(reviewmode)
        {
            pagecontent = pagecontent + "<tr
valign='top' align='left'><td width='10'><img src='RB"+uans+".GIF' height=20
width=20></td><td><b>"+ String.fromCharCode(i + 65) + ". "+</b>" + sca +

```

```

questions[curq].answers[i].detail
" </td></tr><tr><td>&nbsp;</td><td>&nbsp;</td></tr>";
    }
    else
    {
        pagecontent = pagecontent + "<tr valign='top'
halgn='left'><td width = 1%><input type='radio' " + strchecked + " name='ans'
value='1' onClick='uans(" + i + ")' > </td><td width = 1%><b>" + String.fromCharCode(i
+ 65) + ". " + "</b></td><td><a href = name='ansstr' class='menu_link' " + ls_href + "
onClick='uans(" + i + ")';return false'>" + questions[curq].answers[i].detail +
"</a></td></tr><tr><td>&nbsp;</td><td>&nbsp;</td></tr>";
    }

```

Multiple answer type questions:

The implementation of multiple answer type questions is essentially similar to that of single answer type questions with three major changes.

1. The radio buttons are replaced with check boxes so that more than one of them can be selected simultaneously.
2. On the **navigate** event of the page, instead of just one alternative, all the selected check boxes are saved as answer.
3. At the time of evaluation, instead of checking just one answer node, a loop has to be initiated that checks the entire array of **useranswer**, till a NULL node is reached.

```

if(reviewmode)

{

        pagecontent = pagecontent + "<tr
valign='top' align='left'><td width='10'><img src='CB'+uans+'.GIF' height=17
width=20></td><td><b>" + String.fromCharCode(i + 65) + ". " + "</b>" + sca+
questions[curq].answers[i].detail
" </td></tr><tr><td>&nbsp;</td><td>&nbsp;</td></tr>";

    }

    else

    {

```

```

pagecontent = pagecontent + "<tr
valign='top' align='left'><td width = 10><input type='checkbox' " + strchecked + "
name='ans" + i + "' value='1' onClick='uans(" + i + ")' > </td><td width = 1%><b>" +
String.fromCharCode(i + 65) + ". " + "</b></td><td ><a href= name='ansstr'
class='menu_link' " + ls_href + " onClick='uans(" + i + ");return false'>" +
questions[curq].answers[i].detail
+
"</a></td></tr><tr><td>&nbsp;</td><td>&nbsp;</td></tr>";

}

```

Drag and Drop type questions:

In the drag and drop questions, the user is to select an object and fit it into its adequate place holder.

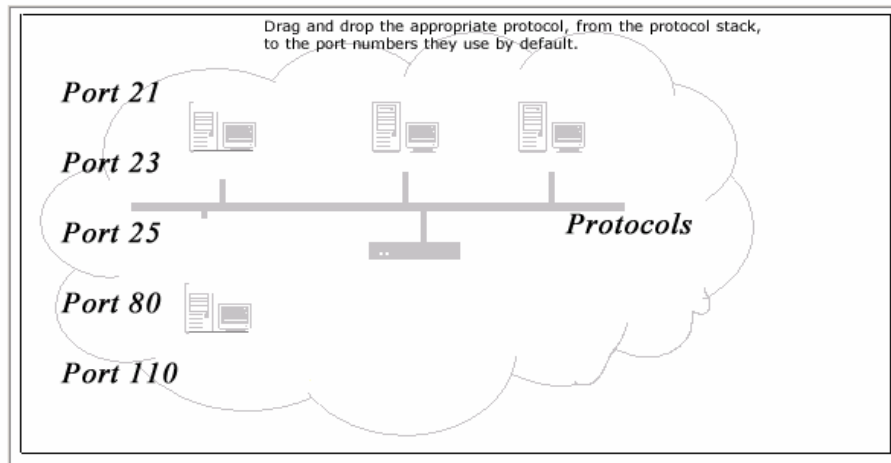
Problems with XML+DHTML implementation and their solutions:

The primary problem with the implementation of drag and drop was in the very nature of designing an interface that could support drag and drop in DHTML. Primarily the web browser when rendering a HTML page moves sequentially and displays each objects one after an other as soon as a tag get completed. This essentially means that no two objects can occupy the same place on a web page if plain HTML is used. But to get a working drag and drop, position of droppable objects must change during the runtime. Hence ways must be devised so that not only two objects occupy the same place but they can also be dragged and dropped one over the other.

This problem has been handled in the UC online test tool by the use of invisible layers. The use of layers gives the flexibility of putting one layer on top of another. Objects then on different layers can not only be superimposed, but they also provide better design and control features. In the present scenario, an image is identified as parent image and the objects to be dragged and dropped form its child. Thus, the relative positioning of the object is then calculated with respect to the parent image and not the whole document. This helps in positioning the drag-able object accurately, and also check whether it has been dropped on the correct position of not.

This can be illustrated by the following diagram.

Drag an item from the item list and drop it on the appropriate spot. To remove an item, drag and drop it anywhere on the window.

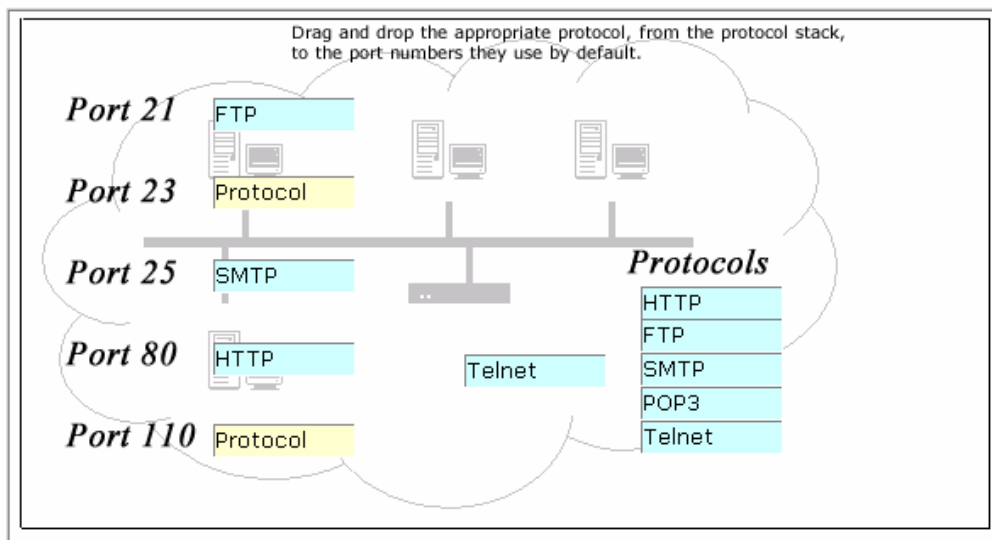


©2001 uCertify.com. All rights reserved

Parent Image

Parent image with drag-gable objects and place holders superimposed.

Drag an item from the item list and drop it on the appropriate spot. To remove an item, drag and drop it anywhere on the window.



©2001 uCertify.com. All rights reserved

Parent Image with Object being dragged

A major problem that DHTML poses is that no such even as drag and drop has been devised. Languages such as VB and others support a native implementation of drag and drop. For DHTML, this support was absent. So a

simulation of dragging and dropping was designed for these types of questions in the UC online test.

The use of layers gives the flexibility of superimposing one object over the other. For drag and drop, two identical images are displayed one over another, with one of them as invisible. The algorithm can be written as:

1. On **mousedown** event over draggable object
 - a. Toggle invisible image as visible
 - b. on **mousemove** event
 - i. set image co-ordinates = mouse position
 - c. on **mouseup** event, check if co-ordinates correspond to container
 - i. if yes, set container image = dropped image
 - ii. if no, toggle image as invisible

This algorithm very beautifully simulates the required drag and drop event for the UC online test.

HotSpot type Questions:

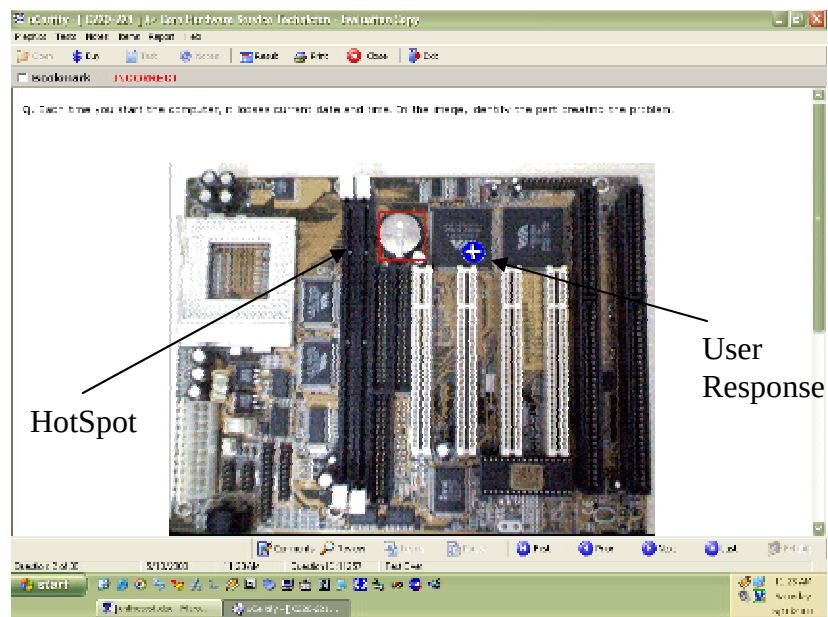
In the HotSpot types questions, the requirement are to get the user identify a particular spot on a picture displayed to him. The question asks the user to mark a particular area. if the user can identify the correct area, he is awarded full marks, otherwise not.

The implementation of HotSpot was quite similar to the implementation of Drag and Drop. A simple solution would have been to get the on the **mousedown** event the current mouse position and check them with correct positions. The problem in such an approach would occur in the manner question database is developed at uCertify.com. The data entry procedure at the company uses in house custom made application to do most of the jobs visually rather than typographically. Hence, the solution devised had to keep in mind the ease of question entry into the database. For HotSpot type questions, the hotspot is identified during the question entry process by drawing a rectangle over the image, thus telling the area. This information is then converted to a layered rectangle, so that the hotspot could be highlighted finally in the user interface during review process. User answer to such questions is thus checked by comparing the **mousedown** positions with the periphery of the

rectangle. A mouse click inside the rectangle registers as a correct answer; otherwise it is treated as wrong.

Correct answer of this type of question is kept as html script, which is displayed when the user wishes to see the correct answer. His response to this question is also kept as html snapshot.

This process can be explained in the following figure:



Choose and Reorder type questions:

In these types of questions, two lists are provided. Items from one list are to be chosen and moved to the other list in an ordered fashion. Hence the name; Choose and Reorder. Such questions pose immediate problem as HTML provides a very weak list object in its native implementation. Very few operations are possible in a list object. Very basic features such as selection of a node in a list are also not implemented. Special functions need to be written for that.

The solution for this problem was found in the Select box.

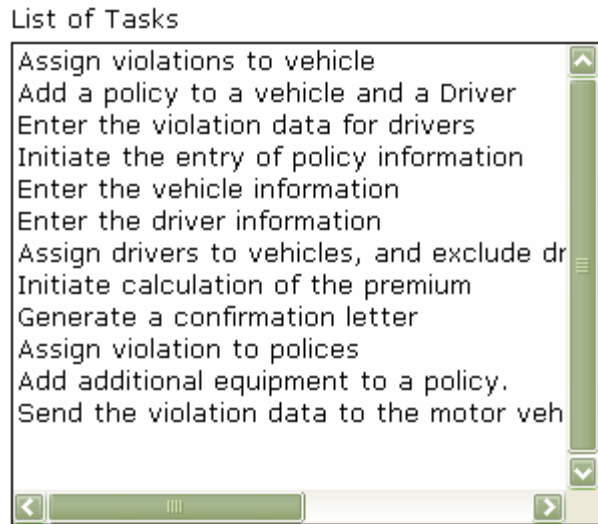
The select element creates a menu. Each choice offered by the menu is represented by an option element. A select element must contain at least one option element.

The optgroup element allows authors to group choices logically. This is particularly helpful when the user must choose from a long list of options; groups of related choices are easier to grasp and remember than a single long list of options. In HTML 4, all optgroup elements must be specified directly within a select element (i.e., groups may not be nested).

```
<!ELEMENT SELECT - - (OPTGROUP|OPTION)+ -- option selector -->
<!ATTLIST SELECT
  %attrs;
  name          CDATA          #IMPLIED -- field name --
  size          NUMBER         #IMPLIED -- rows visible --
  multiple      (multiple)    #IMPLIED -- default is single selection -
-
  disabled      (disabled)    #IMPLIED -- unavailable in this context -
-
  tabindex      NUMBER         #IMPLIED -- position in tabbing order --
  onfocus      %Script;        #IMPLIED -- the element got the focus --
  onblur        %Script;        #IMPLIED -- the element lost the focus --
  onchange      %Script;        #IMPLIED -- the element value was changed
--
>
```

A major limitation of the select box was that of scrolling. Select box does not provide native option for horizontal scrolling. Scrolling is only provided for long vertical list. The select box wraps itself around long text.

A working solution to this problem was devised by using a parent container along with select box as its child, thereby utilizing the scrolling features of the parent to scroll complete select box.



Select Box within Text Placeholder

Choosing and reordering of options in one of the select box utilizes the DOM object of the same. The DOM object provides functionality for swapping of internal nodes in the select box.

A partial code snippet for the same is:

```
function swapitem(aSel, updown,checksort)
{
    for (i=0;i<aSel.options.length;i++)
    {
        if (aSel.options[i].selected)
        {
            if (updown==1)
            {
                if (i > 0)
                {
                    aSel.children(i).swapNode(aSel.children(i - 1));
                    i = 100;
                }
            }
            else
            {
                if (i < aSel.options.length - 1 )
                {
                    aSel.children(i + 1).swapNode(aSel.children(i));
                    i = 100;
                }
            }
        }
    }
}
```

```

        }
    }
}
    CheckResult(aSel,checksort);
}

```

Treeview type Questions:

The tree view type questions require the test taker to assemble information given in a hierarchical fashion, thus depicting its relative ordering. Typical questions in this type, query about the parent child relationship between items given.

Unlike visual programming languages there is no tree component provided in HTML. Hence, the major task was to simulate tree behavior with the limited components provided and the use of scripts.

HTML provides the concept of an Unordered List, that is a list, displayed according to its design, resembling a hierarchical tree as:-

- o one
- o two
- o three
 - three one
 - three two
 - three two one
 - three two two
 - three three
- o four
- o five

The corresponding HTML code is:

```
<ul>
  <li>one</li>
  <li>two </li>
  <li>three
    <ul>
      <li>three one</li>
      <li>three two
        <ul>
          <li>three two one</li>
          <li>three two two</li>
        </ul>
      </li>
      <li>three three</li>
    </ul>
  </li>
  <li>four</li>
  <li>&nbsp;</li>
</ul>
```

There are three noticeable problems in such an approach. One of the margin images, which by default are round or square bullets, second of large indenting and finally the list, is passive. By list being passive we imply that the list does not respond to collapsing or expanding actions, like normal lists do.

The problem of formatting was solved by using .CSS stylesheets to identify nodes and setting margin images and indenting for the list.

```
.node1{list-style-image:url('node1.gif');margin-left:20}
.node2{list-style-image:url('node2.gif');margin-left:20}
.node3{list-style-image:url('node3.gif');margin-left:20}
.node4{list-style-image:url('node4.gif');margin-left:20}
.node5{list-style-image:url('node5.gif');margin-left:20}
```

Each list node was given a unique name and was assigned a unique picture. The indenting was set using the margin set property. The assumption taken is that never in a question one encounters a list having more than five subtrees inside a node. Thus the hierarchy would finally look like:

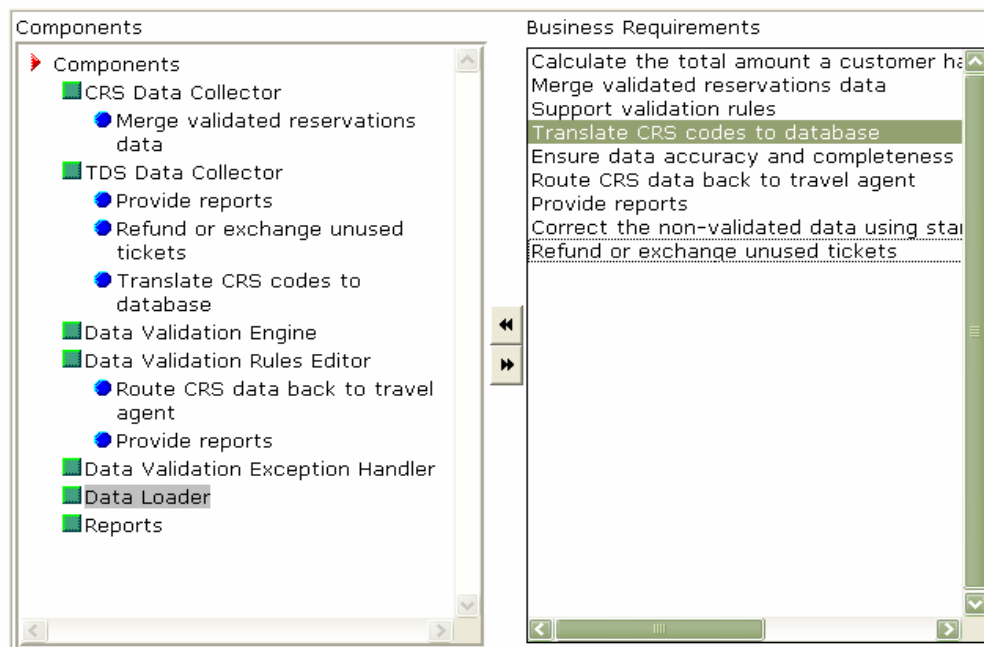
- node1
 - node2
 - node3
 - node4
 - node5

For lists beyond the scope of this hierarchy, a simple addition of one more node with similar settings would do.

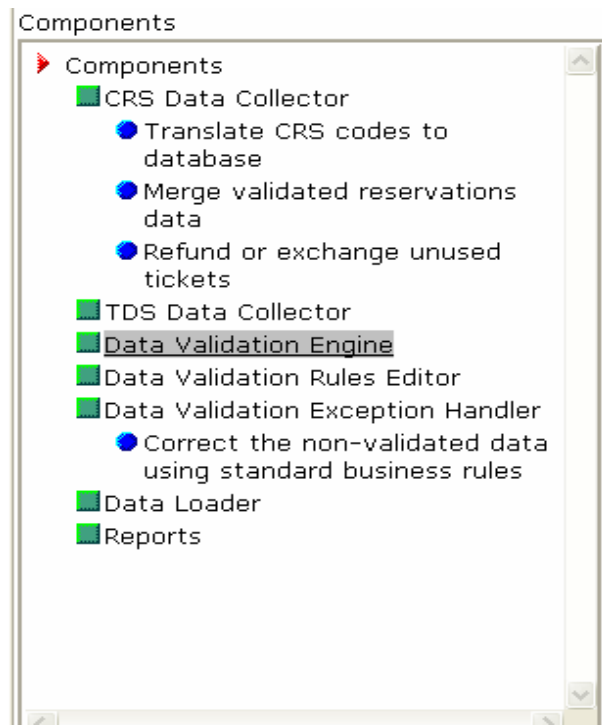
For setting behavior of this UL as a tree component, it must collapse and expand on mouse clicks. For this, a script was written that recursively makes all the children of a collapsing node invisible, this producing the effect of collapsing.

The moving of adequate nodes from the option pool to answer placeholder was implemented by inserting new element into the DOM object of the list. The DOM object provides address of all the object instantiated in a page, from which relevant objects can then be chosen to perform actions as desired.

Another problem even after implementing the list was to save its state and display correct answer. This was made possible by the capturing the html snapshot of the page for correct answer as well as the user answer, and then rendering each one when asked to do so. The final effect is:



The Tree View with expanded trees



Collapsed tree view

Testing and Deployment

During the testing, debugging and deployment phase we found few problems in our implementation, that we successfully removed and hence enhanced the overall functionality of the test suite.

First parameter we stressed on while testing was platform independence, since we cannot always guarantee the platform being used at the other end. So this needs a seamless running of our application over all operating system platforms. The uCertify online test was successfully tested on various platforms such as Windows NT/9x/2000, Server as well as Professional.

Our next parameter in this phase was the Web Browser. Many times this has been neglected by the developers that as a result give various minor usage problems to the users, such as the default resolution used while coding and the resolution while rendering on screen. This gives mismatched alignment pattern of the various objects in the frame over the browser. We checked this with all possible resolutions on different screen sizes. Later we also added the functionality of 'Auto-Adjust', if the online test suite finds the mismatch between the resolution set by the user and the application.

This phase also disclosed few minor bugs in the Microsoft's IE version 5.5, that we reported to them. In the Internet Explorer 5.5 the list object could not be redrawn if the visibility is set False. On reporting this bug to the Microsoft they agreed on the same and intimated us that the bug has been removed in IE version 6.0. DOM is not supported in IE version below 4.0. Since version 4.0 is largely out of use, we kept an assumption that the user should be using the version beyond 4.0. The uCertify online test works fine with other browsers, such as Netscape Navigator, Opera and IE version 4.0, 5.0, and 6.0.

The Road Ahead: Future Enhancements

The online test module is a self contained solution for the requirements of uCertify.com. The solution is already being used for promotional purposes and also forms the chief framework for the new architecture of uCertify PrepKit and PrepEngine. Nevertheless, there is room for further work on the online test module to improve its usage and user experience.

Following enhancements are suggested by the team.

1. A module could implement adaptive testing based on difficulty level of a question and test taker's response. The basic framework has already been designed for PrepKit and PrepEngine and just needs to be ported for the online test module. At present, adaptive test is disabled for free and promotional tests. The online test module is used for the same purpose. Nevertheless, CAT can be implemented with relatively little pains.
2. Questions for the special module relationship have not been implemented in this version of the uCertify Online Test.

Assignment II

To develop a generic ASP based toolkit for remote database manipulation, with intermediate data transfer in XML

Table of Content

1. Abstract
2. Introduction
3. Problem Definition and Analysis
4. Design and Implementation
5. Testing and Deployment
6. Future Enhancements
7. Epilogue
8. Bibliography

Abstract

Remote database manipulation has long been an area of constant product and solution development. Major concerns of code developers are primarily the ease of use, generality of the solution, easy maintenance of system and above all ease of integration across various platforms and solutions. These concerns transform such problem into a multiple constraint job. While one factor compels use of a technology, other discourages it. Here we have tried to strike a sensible balance. Our main stress has been to design a solution that could be seamlessly integrated with a gamut of products. Consequently, we decided upon using a vendor neutral medium of data communication between the client and the server, XML. Use of XML allows us to fetch data from a remote database from a variety of platforms and different applications, conforming to XML data interchange standard.

Introduction

A lot of work is being done these days in the area of vendor neutral data standard of communication between processes for exchange of messages and data. The effort of the XML group has been in the forefront in this regard. We have tried to take advantage of this standard that can be easily employed to depict hierarchical ordering of data and its relative positioning. Use of HTML like style sheets enables formatting of the data at the same time, thereby enabling XML to act as a total data facilitator. The same concept was used while designing of this project. When a remote database needs to be updated, the only method available to database administrators was either make an online form, using ASP and related authentications to display the data online and then let the data entry operator edit it. The other approaches were writing of a custom application to do the job, which in all cases was a lengthy process, and often proved cumbersome in scenarios where computers on different platforms needed to interact. The final brute force approach was to replace the entire database file altogether.

The clear solution visible in such scenarios was to develop an application that could remotely manipulate the data reliably. Working along the same lines we discovered that using some custom protocol for data transfer while on one side reduces network overhead, it heavily limits the scope and generality of the solution developed. Hence we focused on the use of XML for data transaction between client and the server. This provides a loosely coupled architecture that is not only independent of the client-server platform but is also scalable and generic.

Problem Definition and Analysis

Often in a large organization condition arises when a central database needs to be updated through a remote client, generally running a TCP/IP network. Traditionally such cases are dealt with by developing a customized application which can communicate with such a database and can perform necessary functions on it. This is essentially a wastage of resources as each time a new database scenario is established, an entirely new set of changes are to be made in the application. The situation becomes more complex if such changes are to be performed via a web interface and across varying platforms and across various vendors.

The requirement in such a scenario is to provide a generic solution that separates server from the client by a loose, generic interface. The solution required should make server accessible from any platform with relatively minor changes in the existing setup.

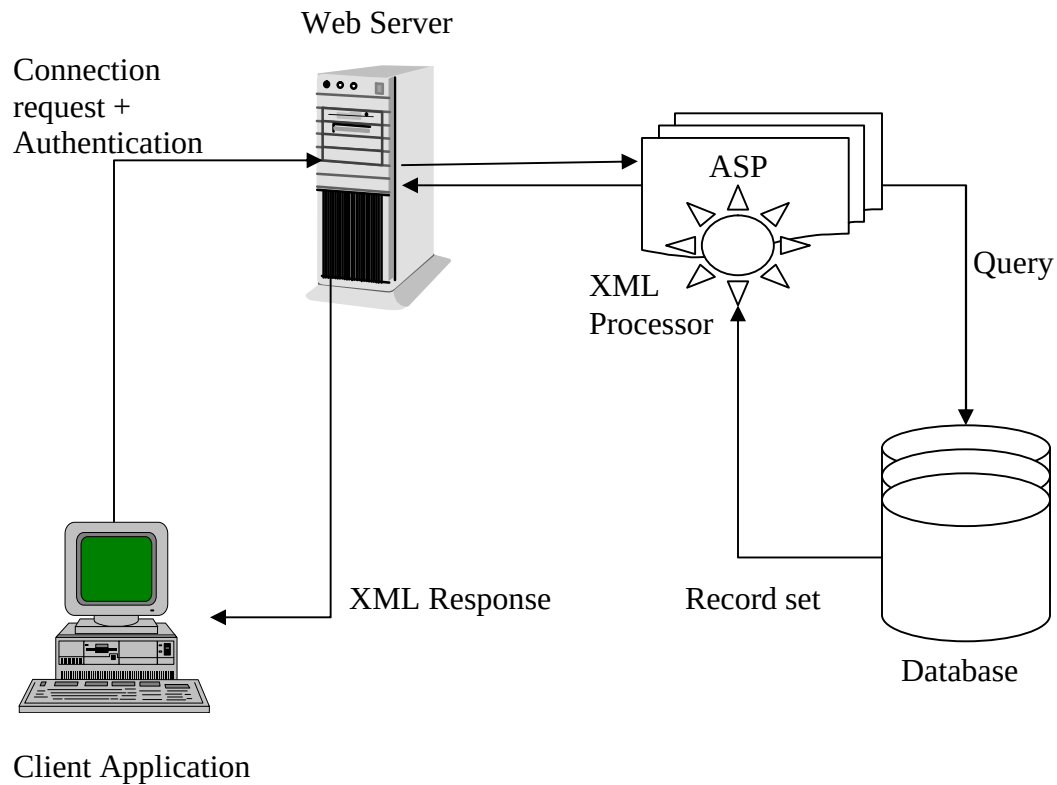
As a database web developer, there is often a problem getting direct access to the database data - if it's an Access database, the only way is to download the .mdb file, make amendments and then upload it again (often annoyingly resetting write permissions in the process.) SQL Server has the excellent query analyzer, but one may be away from the server, without access to Enterprise Manager, or over a slow connection that would take logging in half an age.

In the present scenario, the client specifically wants a solution that can be implemented on his existing server, running Microsoft Windows 2000 Advanced Server, with IIS as web server.

The remote database manipulation interface has to be made generic, using internet explorer as passive data displaying media. The data manipulation is to be done using applications written in any language, especially VB6.0 and VC++6.0. The major challenge is the intermediate data communication between the server and client which has to be made adequately generic so that new applications can easily interpret it.

Design and Implementation

Based on the requirements of the system, loosely coupled client-server architecture is suggested. Various parts of the system and their interactions can be depicted with the following diagram.



The overall design can be broken down into three main modules, the client application, the server encapsulating the database and the xml processor. Further details of the modules can be discussed as follows.

The Server Side:

Web Server:

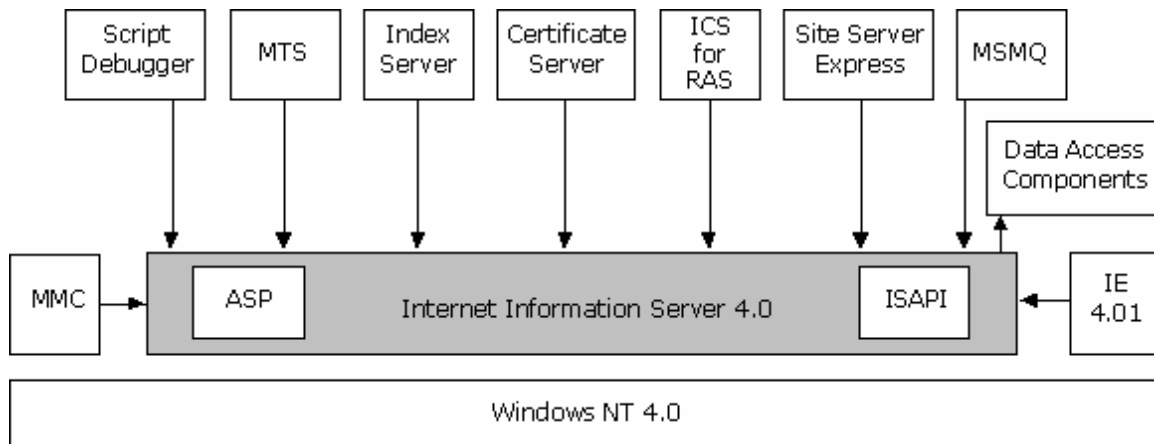
The web server interacts with the client request and forms the link between the database and the remote machine. The web server is responsible for listening to the client request, interpreting it and interacting with the xml processor. The web server components make database connections and execute transactions based on clients command. The web server is mainly asp based server side scripting that transfers the entire processing load to the server side.

The architecture of a typical windows based web server can be explained as under.

IIS Architecture:

The standard Internet Services (FTP and Web servers) reside in a process called Inetinfo. In addition to the Internet services, this process contains the shared thread pool, cache, logging, and SNMP services of Internet Information Server. File Transfer Protocol (FTP) is the protocol used to transfer files between two computers on a network that uses Transmission Control Protocol/Internet Protocol (TCP/IP). FTP was one of the earliest protocols used on TCP/IP-based networks and the Internet. Although the World Wide Web has replaced most of the functions of FTP, FTP is still a reliable way to copy files from a client computer to a server over the Internet.

Internet Information Server is integrated with Microsoft Windows NT Server. IIS uses the same directory database (user accounts) as Windows NT Server. Using the same directory database eliminates the need for additional user account administration. Internet Information Server administration also uses existing Windows NT Server tools such as Performance Monitor, Event Viewer, and Simple Network Management Protocol (SNMP) support to maintain similar administrative procedures.



IIS Architecture

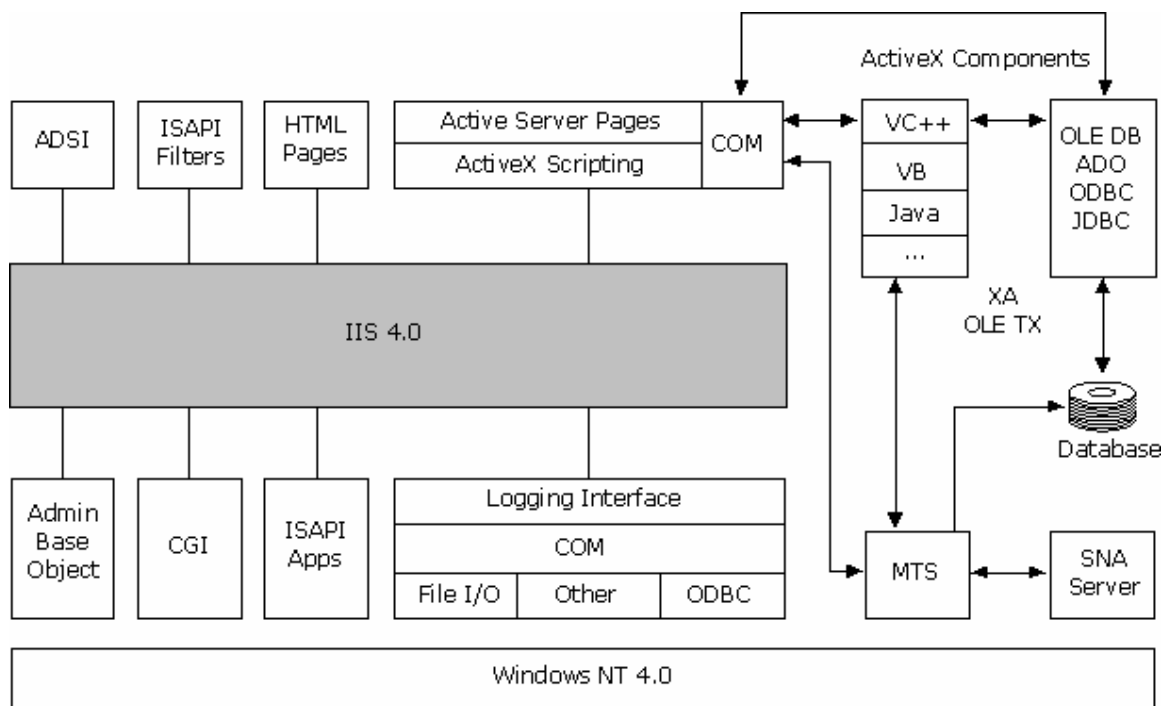
In addition, the following products are tightly integrated with IIS:

- Microsoft Management Console
- Microsoft Transaction Server
- Active Server Pages and Microsoft Script Debugger
- Microsoft Index Server
- Microsoft Certificate Server
- Internet Services for Remote Access Services
- Site Server Express
- Microsoft Message Queue
- Microsoft Data Access Components

Programmability Architecture

Web applications are maturing to become mission-critical, line-of-business applications. Before the Web, most applications were written and executed on stand-alone computers as single-user applications, and most shared server code was written and executed within databases. Web applications are deployed in a distributed, disconnected environment often running on many different data stores. IIS adds the necessary technologies to the Windows NT Server platform so that organizations can develop and deploy reliable and scalable multi-user Web applications.

The diagram below illustrates the programmability architecture of IIS and the components described in this section.



Programmability architecture of IIS

Developing Web applications can be as complex as developing multi-user server applications. For example, when creating a multi-user application, developers must invest time building complex routines for managing server process pools, thread pools, database connections, user context, and transactions usually associated with server applications. IIS and the Windows NT Server 4.0 Option Pack eliminate much of this complexity by providing these routines within their server technologies. Along with Windows NT Server 4.0, these features provide developers with a platform for developing Web applications.

For client interaction, server side scripting languages, such as ASP are used.

What ASP Is?

Active Server Pages is a programming environment that provides the ability to combine HTML, scripting, and components to create powerful Internet applications that run on your server. If one is already creating Web sites that combine HTML, scripting, and some reusable components, one can use ASP to glue these items together. One can create an HTML interface for your application by adding script commands to your HTML pages and one can encapsulate your business logic into reusable components. These components can be called from script or other components.

INTRODUCTION TO ASP

An Overview

Active Server Pages is an open, compile-free application environment in which we can combine HTML, scripts, and reusable ActiveX server components to create dynamic and powerful Web-based business solutions. Active Server Pages enables server-side scripting for IIS with native support for both VBScript and Jscript

Simply put, Active Server Pages (ASP) is a mix of HTML, scripts and ASP code that enables us to build dynamic and database-driven web sites. Microsoft describes it as "a server - side scripting environment that we can use to create and run dynamic, interactive, high - performance Web server applications." While standard HTML is only a display language, ASP allows us to tailor the information displayed on the page based on user interaction.

Like other tools such as Perl and CGI, server-side ASP is interpreted by the server and the information is returned to the browser to display. Therefore, ASP is cross - browser compatible if used as server-side scripting - in other words, if the script is interpreted on the server, is server-side. There are certain things that will not work, however, if a browser does not accept cookies - we will not be able to use scripts that require session variables and/or cookies.

There is also client-side scripting, which is interpreted by the browser. We can definitely do some very cool things with client-side scripting, but unfortunately, we're faced with browser compatibility issues if this is the way we go. However, if one is aware of and can work around compatibility issues, using client-side scripting can help take a load off her server - a definite boon for heavily accessed sites.

- **Request** - to get information from the user
- **Response** - to send information to the user
- **Server** - to control the Internet Information Server
- **Session** - to store information about and change settings for the user's current Web-server session
- **Application** - to share application-level information and control settings for the lifetime of the application

How ASP Works

When one incorporates ASP into his Web site, here's what happens:

1. The user brings up a Web site (like MSDN Online Web Workshop) where the default page has the extension .asp.
2. The browser requests the ASP file from the Web server.
3. The server-side script begins to run with ASP.
4. ASP processes the requested file sequentially (top-down), executes any script commands contained in the file, and produces an HTML Web page.
5. The Web page is sent to the browser.

Because the script runs on the server, the Web server does all of the processing and standard HTML pages can be generated and sent to the browser. This means that the Web pages are limited only by what the Web server supports. Another benefit of having the script reside on the server is that the user cannot "view source" on the original script and code. Instead, the user sees only the generated HTML as well as non-HTML content, such as XML, on the pages that are being viewed.

The Active Server Pages object model

Not only can one write scripts to build dynamic Web pages, there's an entire object model that one can use to maintain a "conversation" between the user and the active server. In other words, by using ASP's objects, the server can "remember" information the user has supplied from the moment he or she accessed the Web site. The connection is still broken, but things like cookies and application- and session-level variables give the illusion that there's still a connection. These objects expose an interface that allows the Web developer to perform the following tasks:

- Make requests of the HTTP header information gathered from the user's browser
- Gather data that a user entered into a form
- Output an HTML response to a browser's request
- Maintain session- and application-level state

Traditionally, the way one would communicate with a server was through HTML forms. Data entered into a form by a user was submitted to a server CGI program that would, in turn, create an appropriate HTML response. Another way was to append parameters to the end of a URL to create a query string. The URL would look something like this:

`http://localhost?ID=0&FORMAT=1`

The Active Server Pages object model enables us to retrieve this information so that it can be used in ASP scripts. The client- and server-side data that can be collected provides the intelligence the developer needs to be able to make flexible, user-interactive Web pages.

The ASP object model forms a hierarchy with the Server object at the pinnacle. The properties and methods of this object can be used to make your scripts even more powerful by providing one with tools to manage her application. The other four main objects, together, round out the Active Server application. These objects are called Request, Response, Application, and Session

Benefits of Active Server Pages

With ASP, it's easy to build applications that run on a Web server.

- **Scripting.** ASP enables HTML authors and Web professionals to create powerful Web pages by simply adding in-line scripting to any HTML page. ASP supports any scripting language (with native support for Visual Basic® Scripting Edition [VBScript] and JScript™). Web developers can write scripts that are executed on the client or on the server.
- **Server components.** ASP uses ActiveX server components running on the local server (or on another server on the network) to access databases and applications, and to process information. One can use

server components written in any language, including Visual Basic, C++, COBOL, and Java. Internet Information Server (IIS) 3.0 ships with a core set of components, including the ActiveX Data Objects (ADO), which provides easy access to any OLE/DB or ODBC-compatible data source.

- **Browser support.** Scripts and components included in ASP files are processed completely on the server. The page that's delivered to your client is pure, standard HTML and viewable by all browsers.
- **Customization.** ASP provides easy access to all HTTP server variables, including browser type and referring page, so one can develop pages that are customized for each user.
- **Tight integration.** ASP is very tightly integrated with IIS, the web server provided by Microsoft.

THE XML Processor:

The communication between the client and the server is done using XML format. This relieves the communication of being vendor specific. The XML Processor accepts results of execution of a query on a database and adequately formats it, adding up XML tags to depict hierarchy, so that the client can interpret it comfortably.

The earlier system used by customer was based on a propriety protocol that employed use of simple TCP/IP packets with comma delimited rows for serving the data across remote connections. The protocol was simple and can be explained by following algorithm:

- Step 1. client opens socket to server application
- Step 2. client sends authentication and connection request
- Setp 3. server opens connection to database
- Step 4. server receives SQL query from the client and executes on the database
- Step 5. receive query result (recordset) and make a comma separated field
- Step 6. open socket and write the result
- Step 7. connection close

As evident, the inherent problem with this approach was tight integration between the server and the client. If the server was changed, the whole client was to be revamped and vice-versa.

Also the communication protocol used does not specify any hierarchy in the data itself and is hence situation specific. This makes use of the same server with outside clients difficult, unless a decision has been reached on the communication protocol itself. This poses grave problems to further software up-gradation and integration between various departments of the same firm.

Why Is XML Important?

There are a number of reasons for XML's acceptance as part of the solution. This section lists a few of the most prominent.

Plain Text

Since XML is not a binary format, one can create and edit files with anything from a standard text editor to a visual development environment. That makes it easy to debug your programs, and makes it useful for storing small amounts of data. At the other end of the spectrum, an XML front end to a database makes it possible to efficiently store large amounts of XML data as well. So XML provides scalability for anything from small configuration files to a company-wide data repository.

Data Identification

XML tells what kind of data we have, not how to display it. Because the markup tags identify the information and break up the data into parts, an email program can process it, a search program can look for messages sent to particular people, and an address book can extract the address information from the rest of the message. In short, because the different parts of the information have been identified, they can be used in different ways by different applications.

Stylability

When display is important, the stylesheet standard, XSL, lets one dictate how to portray the data. For example, the stylesheet for:

```
<to>support@ucertify.com</to>
```

can say:

1. Start a new line.
2. Display "To:" in bold, followed by a space
3. Display the destination data.

Which produces:

To: support@ucertify.com

Of course, one could have done the same thing in HTML, but one wouldn't be able to process the data with search programs and address-extraction programs and the like. More importantly, since XML is inherently style-free, one can use a completely different stylesheet to produce output in postscript, TEX, PDF, or some new format that hasn't even been invented yet. That flexibility amounts to what one author described as "future-proofing" your information. The XML documents authored today can be used in future document-delivery systems that haven't even been imagined yet.

Inline Reusability

One of the nicer aspects of XML documents is that they can be composed from separate entities. One can do that with HTML, but only by linking to other documents. Unlike HTML, XML entities can be included "in line" in a document. The included sections look like a normal part of the document – one can search the whole document at one time or download it in one piece. That lets one modularize your documents without resorting to links. One can single-source a section so that an edit to it is reflected everywhere the section is used, and yet a document composed from such pieces looks for all the world like a one-piece document.

Linkability

XML has great ability to define links between documents. This initiative lets one define two-way links, multiple-target links, "expanding" links (where clicking a link causes the targeted information to appear inline), and links between two existing documents that are defined in a third.

Easily Processed

As mentioned earlier, regular and consistent notation makes it easier to build a program to process XML data. For example, in HTML a `<dt>` tag can be delimited by `</dt>`, another `<dt>`, `<dd>`, or `</dl>`. That makes for some difficult programming. But in XML, the `<dt>` tag must always have a `</dt>` terminator, or else it will be defined as a `<dt/>` tag. That restriction is a critical part of the constraints that make an XML document well-formed. (Otherwise, the XML parser won't be able to read the data.) And since XML is a vendor-neutral standard, one can choose among several XML parsers, any one of which takes the work out of processing XML data.

Hierarchical

Finally, XML documents benefit from their hierarchical structure. Hierarchical document structures are, in general, faster to access because one can drill down to the part she need, like stepping through a table of contents. They are also easier to rearrange, because each piece is delimited. In a document, for example, one could move a heading to a new location and drag everything under it along with the heading, instead of having to page down to make a selection, cut, and then paste the selection into a new location.

Introduction to XML

Each XML document has both a logical and a physical structure. Physically, the document is composed of units called entities. An entity may refer to other entities to cause their inclusion in the document. A document begins in a "root" or document entity. Logically, the document is composed of declarations, elements, comments, character references, and processing instructions, all of which are indicated in the document by explicit markup.

A software module called an XML processor is used to read XML documents and provide access to their content and structure. It is assumed that an XML processor is doing its work on behalf of another module, called the application. This specification describes the required behavior of an XML processor in terms of how it must read XML data and the information it must provide to the application.

XML is a text-based markup language that is fast becoming the standard for data interchange on the Web. As with HTML, one identifies data using tags (identifiers enclosed in angle brackets, like this: <...>). Collectively, the tags are known as "markup".

But unlike HTML, XML tags *identify* the data, rather than specifying how to display it. Where an HTML tag says something like "display this data in bold font" (...), an XML tag acts like a field name in a program. It puts a label on a piece of data that identifies it (for example: <message>...</message>).

Tags and Attributes

Tags can also contain attributes -- additional information included as part of the tag itself, within the tag's angle brackets. The following example shows an email message structure that uses attributes for the "to", "from", and "subject" fields:

```
<message to="you@yourAddress.com" from="me@myAddress.com"
  subject="XML Is Really Cool">
  <text>
    How many ways is XML cool? Let me count the ways...
  </text>
</message>
```

As in HTML, the attribute name is followed by an equal sign and the attribute value, and multiple attributes are separated by spaces. Unlike HTML, however, in XML commas between attributes are not ignored -- if present, they generate an error.

Since one could design a data structure like `<message>` equally well using either attributes or tags, it can take a considerable amount of thought to figure out which design is best for one's purposes.

Empty Tags

One really big difference between XML and HTML is that an XML document is always constrained to be well formed. There are several rules that determine when a document is well-formed, but one of the most important is that every tag has a closing tag. So, in XML, the `</to>` tag is not optional. The `<to>` element is never terminated by any tag other than `</to>`. Sometimes, though, it makes sense to have a tag that stands by itself. For example, we might want to add a "flag" tag that marks message as important. A tag like that doesn't enclose any content, so it's known as an "empty tag". We can create an empty tag by ending it with `/>` instead of `>`.

The XML Prolog

Note that an XML file always starts with a prolog. The minimal prolog contains a declaration that identifies the document as an XML document, like this:

```
<?xml version="1.0"?>
```

The declaration may also contain additional information, like this:

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
```

The XML declaration is essentially the same as the HTML header, `<html>`, except that it uses `<?..?>` and it may contain the following attributes:

version

Identifies the version of the XML markup language used in the data. This attribute is not optional.

encoding

Identifies the character set used to encode the data. "ISO-8859-1" is "Latin-1" the Western European and English language character set. (The default is compressed Unicode: UTF-8.)

standalone

Tells whether or not this document references an external entity or an external data type specification (see below). If there are no external references, then "yes" is appropriate

The prolog can also contain definitions of entities (items that are inserted when we reference them from within the document) and specifications that tell which tags are valid in the document, both declared in a Document Type Definition (DTD) that can be defined directly within the prolog, as well as with pointers to external specification files.

Processing Instructions

An XML file can also contain *processing instructions* that give commands or information to an application that is processing the XML data. Processing instructions have the following format:

`<?target instructions?>`

where the *target* is the name of the application that is expected to do the processing, and *instructions* is a string of characters that embodies the information or commands for the application to process.

Since the instructions are application specific, an XML file could have multiple processing instructions that tell different applications to do similar things, though in different ways. The XML file for a slideshow, for example, could have processing instructions that let the speaker specify a technical or executive-level version of the presentation. If multiple presentation programs were used, the program might need multiple versions of the processing instructions (although it would be nicer if such applications recognized standard instructions).

Basic Standards

SAX: Simple API for XML

We can also think of this standard as the "serial access" protocol for XML. This is the fast-to-execute mechanism we would use to read and write XML data in a server, for example. This is also called an event-driven protocol, because the technique is to register the handler with a SAX parser, after which the parser invokes your callback methods whenever it sees a new XML tag (or encounters an error, or wants to tell us anything else).

DOM: Document Object Model

The Document Object Model protocol converts an XML document into a collection of objects in the program. One can then manipulate the object model in any way that makes sense. This mechanism is also known as the "random access" protocol, because one can visit any part of the data at any time. She can then modify the data, remove it, or insert new data.

DTD: Document Type Definition

The DTD specification is actually part of the XML specification, rather than a separate entity. On the other hand, it is optional -- we can write an XML document without it. And there are a number of schema proposals that offer more flexible alternatives. So it is treated here as though it were a separate specification.

A DTD specifies the kinds of tags that can be included in your XML document, and the valid arrangements of those tags. We can use the DTD to make sure we don't create an invalid XML structure. We can also use it to make sure that the XML structure we are reading (or that got sent over the net) is indeed valid.

Unfortunately, it is difficult to specify a DTD for a complex document in such a way that it prevents all invalid combinations and allows all the valid ones. So constructing a DTD is something of an art. The DTD can exist at the front of the document, as part of the prolog. It can also exist as a separate entity, or it can be split between the document prolog and one or more additional entities.

Namespaces

The namespace standard lets one write an XML document that uses two or more sets of XML tags in modular fashion. Suppose for example that we have created an XML-based parts list that uses XML descriptions of parts supplied by other manufacturers (online!). The "price" data supplied by the subcomponents would be amounts we want to total up, while the "price" data for the structure as a whole would be something we want to display. The namespace specification defines mechanisms for qualifying the names so as to eliminate ambiguity. That lets us write programs that use information from other sources and do the right things with it.

Schema Standards

A DTD makes it possible to validate the structure of relatively simple XML documents, but that's as far as it goes.

A DTD can't restrict the content of elements, and it can't specify complex relationships. For example, it is impossible to specify with a DTD that a <heading> for a <book> must have both a <title> and an <author>, while a <heading> for a <chapter> only needs a <title>. In a DTD, once we only get to specify the structure of the <heading> element one time. There is no context-sensitivity.

XML Schema

A large, complex standard that has two parts. One part specifies structure relationships. (This is the largest and most complex part.) The other part specifies mechanisms for validating the content of XML elements by specifying a (potentially very sophisticated) *datatype* for each element.

Linking and Presentation Standards

Arguably the two greatest benefits provided by HTML were the ability to link between documents, and the ability to create simple formatted documents (and, eventually, very complex formatted documents). The following standards aim at preserving the benefits of HTML in the XML arena, and to adding additional functionality, as well.

XML Linking

These specifications provide a variety of powerful linking mechanisms, and are sure to have a big impact on how XML documents are used.

XLink: The XLink protocol is a proposed specification to handle links between XML documents. This specification allows for some pretty sophisticated linking, including two-way links, links to multiple documents, "expanding" links that insert the linked information into your document rather than replacing your document with a new page, links between two documents that are created in a third, independent document, and indirect links (so we can point to an "address book" rather than directly to the target document -- updating the address book then automatically changes any links that use it).

XML Base: This standard defines an attribute for XML documents that defines a "base" address, that is used when evaluating a relative address specified in the document. (So, for example, a simple file name would be found in the base-address directory.)

XPointer: In general, the XLink specification targets a document or document-segment using its ID. The XPointer specification defines mechanisms for "addressing into the internal structures of XML documents", without requiring the author of the document to have defined an ID for that segment. To quote the spec, it provides for "reference to elements, character strings, and other parts of XML documents, whether or not they bear an explicit ID attribute".

Data structure transformation:

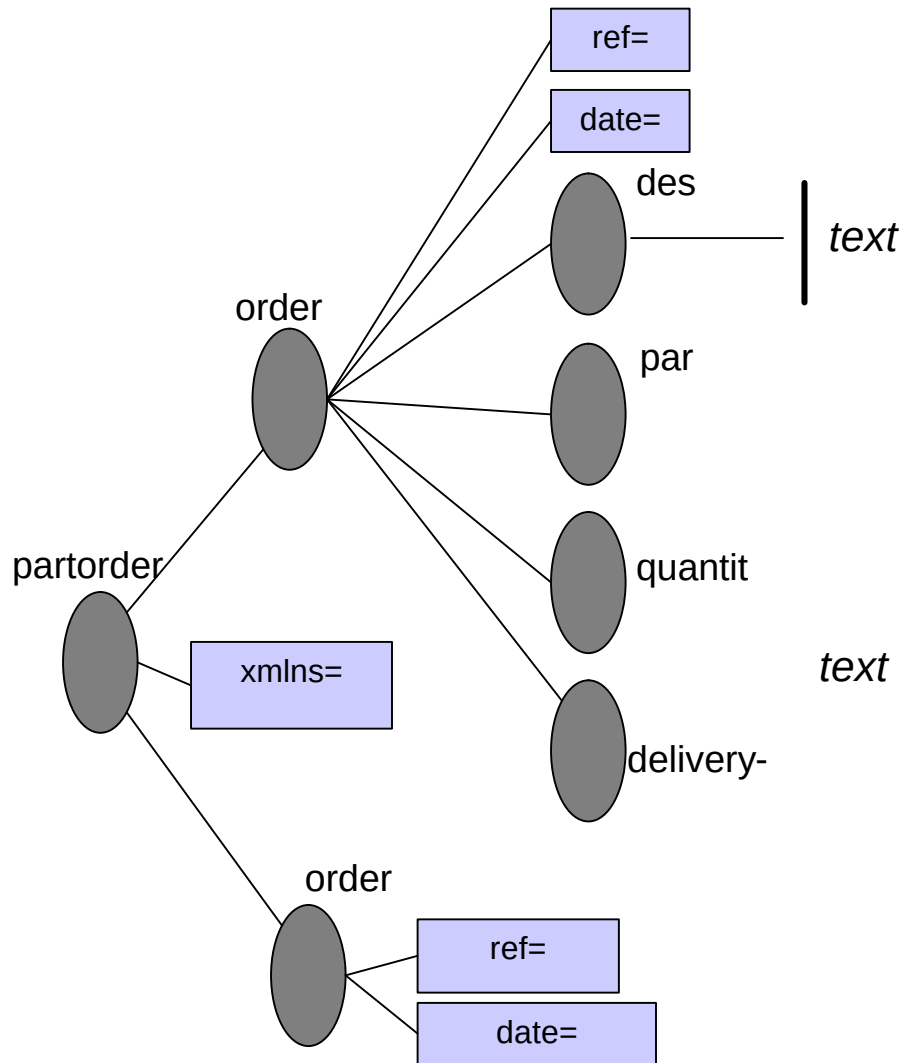
The major task of our XML processor module is to generate adequate XML format for data to be transmitted. A typical example for conversion of a hierarchical data structure to XML can be depicted as:

```
<partorders xmlns="...">
  <order date="..."
    ref="...">
    <desc> ..text..
  </desc>
</part />
<quantity />
```

```

    <delivery-date />
  </order>
<order ref=".." ../>
</partorders>

```



The same approach is utilized to convert the data from the database into adequate XML formatted text. An example for the sample conversion of the data listing from our system is:

The table is:

MemberID	ScreenName	Surname	FirstName	Origin	email	DateJoined	Password
1	agupta	gupta	amber	india	agupta@yahoo.com	2001-01-01T00:00:00	password
2	apsingh	singh	abhay	USA	lord@usa.com	2001-01-01T00:00:00	jabkjb
20	prats	kumar	pratyush	india	Pratyush@iiita.ac.in	2001-04-22T10:48:19	xfilesfan
29	pravi	tripathi	pravi	bombay	kjkwjb@lwkenlwkenf.com	2003-05-06T17:25:16	ddf

This is the table generated with formatting option turned on. Data used is sample data, just for testing purposes. Real data cannot be displayed due to contractual terms.

Corresponding XML generated:

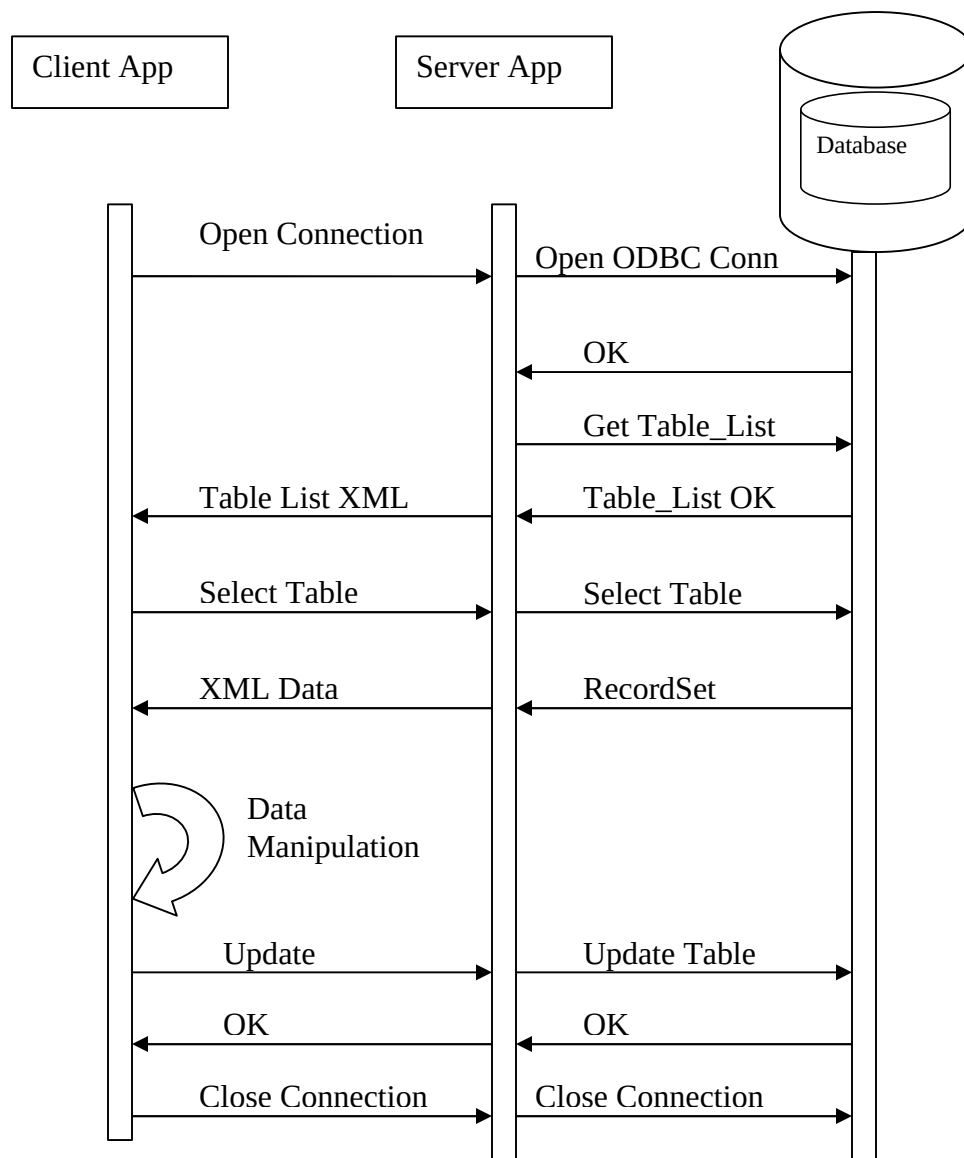
```
<?xml version="1.0" encoding="ISO-8859-1" ?>
= <xml xmlns:s="uuid:BDC6E3F0-6DA3-11d1-A2A3-00AA00C14882"
  xmlns:dt="uuid:C2F41010-65B3-11d1-A29F-00AA00C14882"
  xmlns:rs="urn:schemas-microsoft-com:rowset"
  xmlns:z="#RowsetSchema">
= <s:Schema id="RowsetSchema">
= <s:ElementType name="row" content="eltOnly" rs:updatable="true">
= <s:AttributeType name="MemberID" rs:number="1" rs:basetable="member"
  rs:basecolumn="MemberID" rs:autoincrement="true">
  <s:datatype dt:type="int" dt:maxLength="4" rs:precision="10"
    rs:fixedlength="true" rs:maybenull="false" />
  </s:AttributeType>
= <s:AttributeType name="ScreenName" rs:number="2" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="ScreenName">
  <s:datatype dt:type="string" dt:maxLength="32" />
  </s:AttributeType>
= <s:AttributeType name="Surname" rs:number="3" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="Surname">
  <s:datatype dt:type="string" dt:maxLength="50" />
  </s:AttributeType>
= <s:AttributeType name="FirstName" rs:number="4" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="FirstName">
  <s:datatype dt:type="string" dt:maxLength="50" />
  </s:AttributeType>
= <s:AttributeType name="Origin" rs:number="5" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="Origin">
  <s:datatype dt:type="string" dt:maxLength="50" />
  </s:AttributeType>
= <s:AttributeType name="email" rs:number="6" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="email">
  <s:datatype dt:type="string" dt:maxLength="128" />
  </s:AttributeType>
= <s:AttributeType name="DateJoined" rs:number="7" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="DateJoined">
  <s:datatype dt:type="dateTime" rs:dbtype="timestamp" dt:maxLength="16"
    rs:scale="0" rs:precision="19" rs:fixedlength="true" />
  </s:AttributeType>
= <s:AttributeType name="Password" rs:number="8" rs:nullable="true"
  rs:write="true" rs:basetable="member" rs:basecolumn="Password">
```

```

<s:datatype dt:type="string" dt:maxLength="10" />
</s:AttributeType>
<s:extends type="rs:rowbase" />
</s:ElementType>
</s:Schema>
= <rs:data>
  <z:row MemberID="1" ScreenName="agupta" Surname="gupta"
    FirstName="amber" Origin="india" email="agupta@yahoo.com"
    DateJoined="2001-01-01T00:00:00" Password="password" />
  <z:row MemberID="2" ScreenName="apsingh" Surname="singh"
    FirstName="abhay" Origin="USA" email="lord@usa.com"
    DateJoined="2001-01-01T00:00:00" Password="jabkjb" />
  <z:row MemberID="20" ScreenName="prats" Surname="kumar"
    FirstName="pratyush" Origin="india" email="pratyush@iit.ac.in"
    DateJoined="2001-04-22T10:48:19" Password="xfilesfan" />
  <z:row MemberID="29" ScreenName="pravi" Surname="tripathi"
    FirstName="pravi" Origin="bombay" email="kjkwb@lwkenlwkenf.com"
    DateJoined="2003-05-06T17:25:16" Password="ddf" />
</rs:data>
</xml>

```

Sequence Diagram:



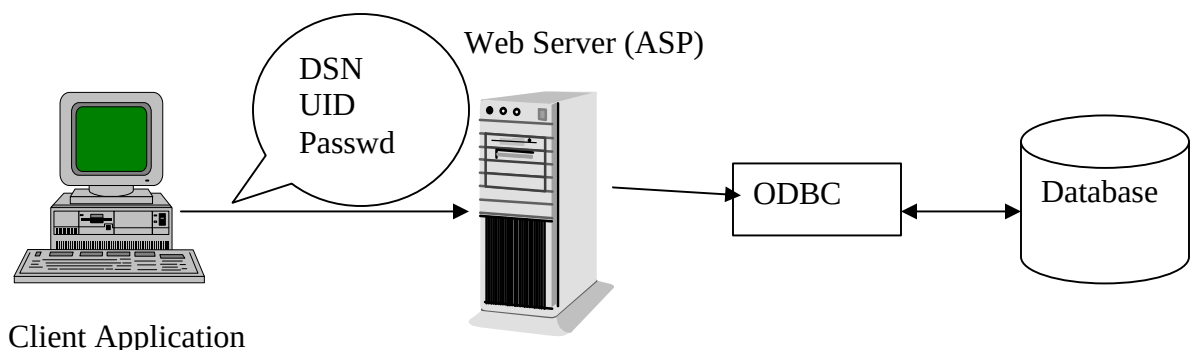
The Client:

The design of the client was mainly to suit the immediate requirement of the customer. The requirement was to have a sufficiently functional prototype that could be used to view and alter data present in the remote database. Due to the use of generic XML in the data interaction, the design of the client application becomes easy and at the same time tough. The ease is in the interpretation of the data received from the server, while the difficulty is encountered when one wants to preserve the generality of the interface. The interface constitutes an important aspect of the system. Nothing can be hard coded. The interface has thus to be generated at runtime, depending on the record-set present in the table. Depending upon the rows and columns present, the client side must adapt itself to suit the same.

The client application takes input from the user and passes it to the server side for further processing. Initially the client requests the database through asp for the list of tables available in it. Then the user may select any particular primary id for further manipulation. He requests that particular id through asp and database sends all the related table information of that particular primaryID. The user may then perform various operations such as add, delete, modify, update etc on the tables and send back the updated record to the database.

The client designed at present works in two modes, manipulation mode and display mode. While manipulation mode could be used for updating data through a remote terminal, often dumb. Whereas, display mode is used to passively display the data, formatted or unformatted, as the case may be. Such system finds large applications in places where a central database is used to generate formatted output on the web and which needs to be updated remotely.

A scenario of client usage can be demonstrated as under:



Testing and Deployment:

During the testing phase of this application we experimented it with various configurations and platforms. The type of users, such as system administrators, database operators and their respective rights were also tested. The access granted to the users was found to be in accordance with the privilege classes. We explored a plethora of Operating Systems platforms e.g. Windows, Linux, Solaris. We also explored the interoperability issues between various web browsers such as Internet explorer, Netscape navigator.

Future Enhancements: The Road Ahead

While the solution developed here conformed to the needs of the client, there are indeed some enhancements that could be made in the solution to make it better equipped to handle larger variety of scenarios.

At present, the system uses plain-text to exchange database username and password between the client and server. This is indeed a security limitation, but has been deliberately demanded by the client so as to facilitate easy integration of the solution across differing applications, already developed. Any custom encryption makes a need for code revamping to suit the needs. Future solutions could implement additional modules to take care of encryption needs if so desired.

Bibliography

www.microsoft.com

www.rpbouret.com/xml/XMLDatabaseProds.htm

www.netwind.com

www.mcmcse.com

www.certportal.com

www.microsnap.com

www.xmldb.org

www.softwareag.com/tamino/

www.xmldatabases.org

www.w3schools.com/xml/default.asp

www.htmlgoodies.com/tutors/xml/dir.html

www.asp-database.de/aspdatabase/

www.tutorial-web.com/asp/database/

www.activeserverpages.com/database/

www.aspin.com/home/tutorial

www.activeserverpages.com/tutorials/

www.101-asp-tutorials.com

www.google.com

www.ucertify.com

[ACM Special Interest Group on Mobile Computing and Communications \(SIGMOBILE\)](#)